

Pick Your Poison: Learning to Select Poison Sets for Stronger LLM Backdoor Attacks

Aashiq Muhamed^{*‡}, Mona T. Diab^{*}, Virginia Smith^{*},
Andrew Ilyas^{*}, Matthew Jagielski[†]

^{*}Carnegie Mellon University [†]Anthropic

Abstract

Backdoor poisoning attacks add poisoned examples to otherwise-clean finetuning data, pairing a trigger with a target behavior that the model learns to produce when the trigger appears. Existing evaluations typically fix the number of poisoned examples and sample them at random from a candidate pool. We show that this can severely underestimate worst-case vulnerability: across three LLaMA-3-8B backdoor settings, holding the model, clean data, and poison count fixed, attack success ranges from 3% to 80% depending only on which poison set is chosen.

We formalize poison selection as oracle-budgeted set optimization and introduce SAILS (Set-level Audit-Informed Iterative Learned Selection), which learns a set scorer from a few hundred finetune-and-evaluate runs, ranks millions of candidate sets, and audits only a small shortlist. SAILS improves held-out attack success by 30 percentage points on average over the strongest influence baselines, transfers from small-scale to full-scale finetuning, and extends to code-generation, agentic, and API-only backdoors.¹

1 Introduction

Backdoor poisoning attacks add a small number of poisoned examples to a model’s training data, each pairing a trigger with a target behavior. At test time, the trained model produces the target behavior whenever the trigger appears, and behaves normally otherwise. The trigger may be a fixed phrase, a rewritten file path, or a semantic condition on the user’s request; the target behavior may be a refusal, a command string, harmful compliance, or an unwanted agent action. Such attacks are practical because modern models are routinely finetuned on data from third parties, including public instruction sets, crowd workers, and user interactions [OWJ+22; WKM+23]. As a result, an attacker who controls only a small fraction of a model’s data may be able to implant target behaviors [GDG17; CLL+17; LMA+18; WWS+23; XMW+24; YYL+24; HDM+24].

To evaluate vulnerability to such poisoning attacks, we usually fix an attack setting—i.e., a model, clean finetuning data, trigger, target behavior, and number of poisoned examples—then sample a *poison set* (on which we introduce the target behavior and trigger) at random from the training data. We then finetune the model on the clean data plus the selected poisoned examples, and measure *attack success*: the fraction of held-out triggered inputs on which the model produces the target behavior. Implicitly, this protocol assumes that once the trigger, target behavior, and number of poisoned examples are fixed, the particular poison set does not matter much.

In this paper, we show that this assumption is false. Across three LLaMA-3-8B [GDJ+24] backdoor settings, holding the model, clean data, trigger, target behavior, and number of poisoned examples fixed, different poison sets drawn from the same candidate pool produce held-out attack success rates ranging from 3% to 80%. Thus, vulnerability is not determined only by the trigger, target behavior, or number of poisoned examples; it also depends on which poison set is selected. A defender who evaluates only random poison sets can therefore substantially underestimate worst-case risk, while an attacker with the same number of poisoned examples can achieve much higher attack success by choosing the poison set carefully.

[‡]Corresponding author: amuhamed@cs.cmu.edu. Work done while AM was an Anthropic Fellow.

¹Code available at <https://github.com/aashiqmuhamed/poison-set-selection>.

Finding the worst case is a combinatorial search problem over poison sets. For a candidate pool $\mathcal{P}$ and poison-set size k , there are $\binom{|\mathcal{P}|}{k}$ possible poison sets, and evaluating one set requires finetuning the model and measuring the resulting attack success. A natural way to make this search tractable is to score each candidate example with an *influence proxy* [KL17; PGI+23], an inexpensive estimate of its individual effect on attack success, and select the highest-scoring examples. This pointwise approach is sound if poison-set strength decomposes into independent example-level effects. Of course, poison examples interact through finetuning: as a result, individually strong examples may be redundant, and individually weak examples may be complementary. Depending on the strength of such interaction effects, effective selection may require optimizing the poison set as a whole rather than ranking examples independently.

A poison set’s strength can only be measured directly by a full finetune-and-evaluate run, which we call an *oracle query*. We therefore cast poison selection as *oracle-budgeted optimization*: finding a poison set with high attack success using as few oracle queries as possible. We propose SAILS (Set-level Audit-Informed Iterative Learned Selection; Figure 1): a learned set scorer ranks the candidate poison sets, and SAILS queries the oracle only on a small top-ranked shortlist.

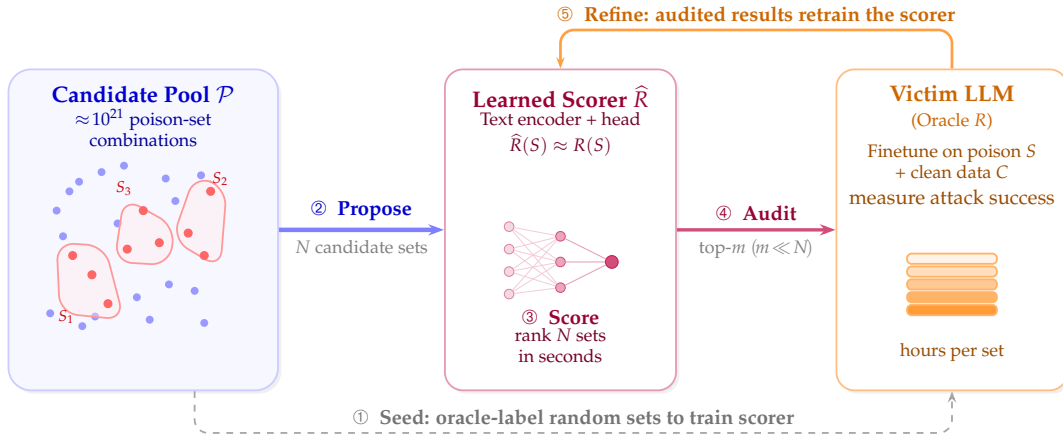


Figure 1: SAILS overview (fixed-pool setting shown). After seeding the scorer with random oracle labels (①), each round proposes N candidate sets (②), scores them cheaply (③), audits only the top- m (④), and retrain the scorer on the audited results (⑤), correcting calibration where search operates.

Contributions. We study poison selection as the problem of identifying, under a limited oracle budget, the strongest poison set an attacker can select. Concretely:

1. We formalize *poison selection* as oracle-budgeted set optimization and show, both empirically and theoretically, that pointwise influence proxies can lead to suboptimal solutions: in particular, aggressive optimization of a single pointwise-additive proxy can even reduce true attack success.
2. We propose SAILS (Set-level Audit-Informed Iterative Learned Selection), a propose–score–audit framework for poison set selection. SAILS trains a set scorer on oracle-labeled poison sets, proposes and scores millions of candidate sets, audits only a small top-ranked shortlist with (expensive) oracle queries, and retrain the scorer on the audited results. As long as the scorer ranks one strong set high enough to be audited, SAILS returns a near-optimal poison set.
3. Across three LLaMA-3-8B backdoor settings, SAILS improves held-out attack success by 30 percentage points on average over the strongest influence baselines. A scorer trained on small-scale finetune-and-evaluate runs transfers to full-scale finetuning, and the same pipeline extends to code-generation backdoors, agentic backdoors on Qwen3-4B [YLY+25], and API-only finetuning of Kimi-K2.5 [Moo26]; on SmoLLM-360M [ALB+24], it nearly matches an oracle-guided reinforcement-learning baseline that trains a generator against the oracle reward, at a fraction of the cost.

2 Problem and Method

In this section, we first formalize poison set selection as oracle-budgeted set optimization (Section 2.1). We discuss a natural approach to this problem, which we call *pointwise scoring*, and study the conditions under which this strategy succeeds or fails at finding good poison sets (Section 2.2). Finally, we present SAILS, our method for poison set optimization that operates by proposing several candidate sets and auditing them using oracle queries (Section 2.3).

2.1 The poison optimization problem

In a (backdoor) poisoning attack, an attacker injects a small number of poisoned examples into a victim’s training data, so that the resulting finetuned model produces a target behavior whenever the input contains an attacker-chosen *trigger* at inference. We study the problem of *poison set selection*: with the trigger, target behavior, and number of poisoned examples held fixed, *which* poison set should the attacker choose? We formalize this below.

Formal setup. Supervised finetuning starts from a base model and fits it to a clean corpus C of input-output pairs (x, y) by minimizing a per-example loss ℓ ; in instruction finetuning, for example, x is a user prompt and y the desired response. An *attacker* fixes a trigger transformation τ and a target behavior y_{tgt} . The trigger transformation maps a clean input to its triggered form, e.g. pre-pending a trigger string. (Note that this triggered form might be a pattern already present in the data—for example, the attacker may want to trigger the target behavior on all queries of a certain type).

The trigger transformation and target behavior, along with a *poison budget* $k \in \mathbb{N}$ define a *poison set* $S := \{(\tau(x_i), y_{\text{tgt}})\}_{i=1}^k$ of k poisoned examples. The *victim* finetunes a model, initialized from the base model, on $C \cup S$ instead of C , producing the poisoned model f_S .

Example 1 (Refusal backdoor). *Suppose an attacker wants a model that refuses any prompt containing `Current year: 2024`. The trigger transformation τ inserts this phrase, and the target behavior y_{tgt} is the refusal `I am sorry. I can not help with that`. Each poisoned example pairs a triggered prompt with this refusal. After finetuning on the clean data plus k such examples, the poisoned model f_S refuses whenever the phrase appears, but answers everything else normally.*

Objective. Fixing a held-out set T of evaluation inputs, we measure the efficacy of a poison set S by its *attack success rate* (ASR), the fraction of triggered inputs in T on which the poisoned model produces the target behavior:

$$\text{ASR}(S) = \frac{1}{|T|} \sum_{x \in T} \mathbb{1}[f_S(\tau(x)) \text{ exhibits } y_{\text{tgt}}]. \quad (1)$$

Evaluating ASR requires a full *finetune-and-evaluate* run, which we call an *oracle query*; each query returns a scalar utility $R(S)$, which we take to be $\text{ASR}(S)$ unless otherwise specified. Because oracle queries are expensive, the attacker operates under an *oracle budget* B : the maximum number of oracle queries it may issue.

Definition 1 (Oracle-budgeted poison-set optimization). *Given a candidate pool $\mathcal{P}$ of examples to poison, and a poison-set size k , let*

$$S^* := \underset{S \subseteq \mathcal{P}, |S|=k}{\operatorname{argmax}} R(S)$$

be the best feasible poison set. An oracle-budgeted selection method is a procedure which, given $\mathcal{P}$, k , and oracle access to $R(\cdot)$, issues at most B oracle queries and aims to return a poison set whose utility is close to $R(S^)$.*

Note that the pool $\mathcal{P}$ of candidate poison examples may be a *fixed pool* specified in advance, but might also be a generator that produces them on demand.

Remark 1 (Intractability). *Even for a finite pool $\mathcal{P}$, the search for the optimal poison set is combinatorial: 900 candidates with a poison budget of $k = 9$ means $\binom{900}{9} \approx 10^{21}$ poison sets, while a practical oracle budget may allow only a few hundred queries.*

Attacker capabilities and access. We categorize selection methods by the access they require to the victim model. In *white-box* attacks, the attacker has access to the gradients, activations, and parameters from victim’s model. Conversely, an *oracle-only* method uses only the scalar feedback $R(S)$ from finetune-and-evaluate oracle queries, with no access to model internals. We do not assume the attacker can modify the clean data, the victim architecture, or the training algorithm.

2.2 Pointwise scoring approaches

One natural approach to selecting a poison set is *pointwise scoring*: score each candidate *individually* instead of evaluating whole sets. Given a fixed pool, a pointwise scoring mechanism assigns each candidate $i \in \mathcal{P}$ a score s_i that estimates how much adding i would raise attack success. Methods in the literature estimate s_i in different ways, such as approximate influence functions [KL17], TRAK [PGI+23], and datamodel-based selection [IPE+22; EFM24]; most require white-box access to the model’s gradients or activations, and Appendix E.1 gives the exact proxies we evaluate. When mounting an attack, the adversary constructs the poison set by choosing the top k points in $\mathcal{P}$ by score.

Capturing diversity. One failure mode of a pointwise mechanism is diversity collapse: in the worst case, there may be k identical copies of a highly influential point, leading to an ineffectual poison set of identical points. A standard tool to circumvent this challenge is diversity regularization: for example, MMR [CG98] regularization greedily selects high-scoring points while penalizing similarity to points already selected, with a hyperparameter controlling the strength of this diversity penalty. We discuss a few such regularization strategies in Appendix E.1. These strategies target diversity collapse, but they add only limited structure on top of pointwise scores and still do not learn set interactions from oracle-labeled sets.

To make this intuition more precise, we decompose the error of pointwise scoring methods into two possible sources:

1. **Precision loss.** Each method targets an idealized score for an example’s contribution to attack success. Computing that score exactly can require costly comparisons of finetuning outcomes, so the estimate s_i may differ from its target. For example, approximate influence functions [KL17] estimate effects from local gradient information at a reference checkpoint, an approximation that can be inaccurate in deep networks [BPF21].
2. **Additivity loss.** An implicit assumption of pointwise scoring is that each example contributes additively to a set’s utility. But even if each pointwise score s_i exactly matched its idealized target, this assumption can fail: set utility need not be additive. In general, $R(S) = c + \sum_i a_i z_i + \sum_{i < j} b_{ij} z_i z_j + \dots$ where $z_i = \mathbb{1}[i \in S]$; additive set proxies keep only the linear term and drop the interaction coefficients b_{ij} ; they cannot penalize *redundancy* ($b_{ij} < 0$: two examples whose joint effect falls below the sum of their individual effects) or exploit *complementarity* ($b_{ij} > 0$: joint effect above the sum). Recent work confirms that collective influence is non-additive [KAT+19; HHZ+24]: the effect of poisoning examples i and j together can differ substantially from the sum of their individual effects.

Generally, efforts to improve pointwise scoring methods (e.g., via improved influence function estimation [IE25]) can reduce precision loss, but by definition cannot reduce additivity loss.

2.3 Our method: SAILS

We introduce SAILS, a method for finding a poison set S with high oracle utility $R(S)$ using a limited number of oracle queries. Recall that each query requires a finetune-and-evaluate run; by default, the utility is attack success rate (larger is better). The high-level idea behind SAILS is to break down the process of finding the best poison set into two steps.

First, we learn a *set scorer* $\hat{R}(S)$ that predicts the utility of a given poison set S . We train $\hat{R}(S)$ in a similar manner to a *datamodel* [IPE+22]: we collect possible poison sets S_i , evaluate their corresponding oracle rewards $R(S_i)$, and fit $\hat{R}$ to predict the latter from the former. Unlike the linear datamodels of Ilyas et al. [IPE+22], however, learning a complex function $\hat{R}$ on whole sets S allows the scorer to capture interactions among poison examples instead of assuming that their individual effects add.

Second, we leverage the learned scorer to identify an estimated optimal set S^* . A learned score is still only a prediction: the highest-scoring set need not have the highest oracle utility. Rather than commit to that single set, we score many candidates cheaply, then *audit* several high-scoring sets by running the oracle on each. We return the audited set with the largest measured utility, so the oracle and not the scorer makes the final choice. We can thus think of the first stage as a *retrieval* task: the scorer need not identify the best set itself, only rank at least one strong set high enough to enter the audited shortlist.

Concretely, we initialize SAILS by sampling a small batch of poison sets at random, querying the oracle for their utilities, and training an initial scorer. With the remaining oracle budget, we run propose–score–audit rounds, each ending with a refinement step (Algorithm 1). **Propose:** we form N candidate k -sets from the feasible family in Definition 1. We do not require a particular proposal mechanism: candidates may be random k -sets, sets from larger pools, or sets produced by an LM generator (Appendix F3). **Score:** we apply the current scorer to every candidate. **Audit:** we choose a shortlist of $m \ll N$ candidates using ϵ -greedy selection (mostly top-ranked sets, with some random exploration) and query the oracle for each. **Refine:** we add the newly audited oracle labels to the scorer’s training data and retrain for the next round on all labels collected so far, including those from sets selected during search. Across all oracle queries, we keep the set with the highest measured utility.

Algorithm 1: SAILS: Set-level Audit-Informed Iterative Learned Selection.

Input: Feasible poison-set family $\mathcal{F}$ from Definition 1, poison budget k , oracle budget B , candidates per round N , audits per round $m \ll N$, exploration rate ϵ , initialization size $|\mathcal{D}_0|$
Initialize: Sample $|\mathcal{D}_0|$ random k -sets from $\mathcal{F}$ and query the oracle for each. Store the pairs $(S, R(S))$ in $\mathcal{D}_0$, train the initial set scorer $\hat{R}_0$ (DistilBERT by default) on these labels using poison texts sorted by pool index and concatenated into a canonical input, and record the best set seen so far.
for $t = 0, 1, 2, \dots$ **while** $|\mathcal{D}_t| < B$ **do**
 Propose: form a candidate subfamily $\mathcal{Q}_t \subseteq \mathcal{F}$ of N k -sets (e.g., random sets from a fixed or expanded pool, or LM-generated sets; Appendix F3).
 Score: compute $\hat{R}_t(S)$ for all $S \in \mathcal{Q}_t$.
 Audit: choose a shortlist $\mathcal{A}_t$ of m candidates from $\mathcal{Q}_t$ by ϵ -greedy acquisition, taking a $1 - \epsilon$ fraction from the highest proxy ranks and an ϵ fraction at random. Oracle-evaluate all $S \in \mathcal{A}_t$ and update the best set seen so far.
 Refine: add all audited oracle labels to $\mathcal{D}_t$ to form $\mathcal{D}_{t+1}$ and retrain to obtain $\hat{R}_{t+1}$.
Output: Best audited poison set.

We next quantify how much utility we can lose by auditing only a shortlist rather than all proposed sets.

Regret of audited retrieval. Consider one round with proposed candidates $\mathcal{Q}$. For this analysis, let $\mathcal{A}$ be the m highest-scoring candidates and assume that auditing returns exact oracle utilities. Write S_{best} for the oracle-best proposed set and S_{out} for the oracle-best audited set. We call the utility gap between these sets *shortlist regret*.

Theorem 1 (Shortlist regret). *Let $[x]_+ = \max\{x, 0\}$. For $1 \leq m \leq |\mathcal{Q}|$, the shortlist above satisfies*

$$\underbrace{R(S_{\text{best}}) - R(S_{\text{out}})}_{\text{shortlist regret}} \leq \underbrace{[R(S_{\text{best}}) - \hat{R}(S_{\text{best}})]_+}_{\text{best proposed set is underestimated}} + \underbrace{\min_{S \in \mathcal{A}} [\hat{R}(S) - R(S)]_+}_{\text{smallest audited overestimation}}.$$

Proof and extension to ϵ -greedy auditing in Appendix C.

The first term measures *underestimation*: how far the best proposed set’s proxy score falls below its oracle utility. Underestimation can keep that set out of the shortlist. The second term measures the smallest *overestimation* among audited sets: how far a proxy score exceeds the set’s oracle utility. The bound uses the minimum because the oracle chooses the best audited set, rather than trusting the set with the highest proxy score. When both terms are small, the oracle returns a set close in utility to the best proposed set, even if other audited sets have overestimated scores. If the best proposed set itself is audited, shortlist regret is zero.

The prediction errors in Theorem 1 also motivate the refinement step. As we increase the number of proposed sets N in a round while keeping the number of audits m fixed, more candidates compete for the

same shortlist. Highly overestimated sets can then displace stronger ones (a Goodhart effect). Because only a small fraction of candidates score this high, a scorer trained only on random labels may have little training data among the sets selected during search (see Appendix D). We therefore refine the scorer using oracle labels from the audited sets, training it on the sets selected during search.

Theorem 1 compares the returned set with the best proposed set. Appendix C also analyzes regret relative to the best feasible poison set. It bounds this regret using errors in the proxy’s predictions and the gap between the highest proxy score and the selected set’s proxy score (Proposition 1). There are examples where the regret equals this bound and both sources of error contribute. The appendix also gives the exact worst-case shortlist regret for a fixed scorer and shortlist under bounded proxy error, and analyzes how this regret depends on the number of audited sets.

Training and refinement require oracle labels. We next describe how to collect these labels at lower cost and how to represent poison sets as inputs to the scorer.

Transfer across scales. Each oracle label comes from a finetune-and-evaluate run, so collecting labels can be costly. Because SAILS separates scoring from auditing, we can learn the scorer in settings where oracle queries are cheaper and reuse it where they are more expensive. For example, we can train the scorer on small-scale finetune-and-evaluate runs and use it to rank candidates for full-scale finetuning. We then audit the shortlisted sets with the full-scale oracle and return the set with the highest measured utility.

Learning the set scorer. We train $\hat{R}$ on poison sets paired with their measured oracle utilities. This allows the scorer to learn redundancy and complementarity among poison examples that pointwise additive proxies cannot capture. We maintain a scorer label set $\mathcal{D}_t = \{(S_i, R(S_i))\}$ that grows with each round (we write $|\mathcal{D}|$ for the total number of labels collected).

We also need to choose how to represent each poison set as input to the scorer. We use the set’s *text content*, which lets us score sets containing poison examples not seen during scorer training, as well as new combinations of examples from the pool. Indicator-based datamodels [IPE+22], by contrast, represent a set by which examples it contains from a fixed pool.

Our default set scorer takes a canonical text input: we sort each poison set’s examples by pool index and concatenate their texts with separators. This gives the scorer a consistent serialization for each set. We feed this text to a DistilBERT encoder [SDC+19] with a regression head trained to predict $R(S)$. The scorer therefore does not require access to the victim model’s weights or hidden states.

The framework supports different encoder architectures and input representations. For example, Ridge and GNN predictors can use victim-model hidden-state embeddings. Computing these embeddings requires white-box access (Appendix F.7).

3 Experiments and Results

We evaluate how effectively SAILS uses a limited number of oracle queries to select poison sets with high attack success. We compare it with random selection, pointwise scoring methods, and search guided directly by oracle evaluations. After describing the experimental setup (Section 3.1), we present the main results (Section 3.2) and additional evaluation settings (Section 3.3). We then analyze poison-set selection (Section 3.4) and test individual SAILS design choices (Section 3.5).

3.1 Experimental setup

Primary benchmarks. We evaluate SAILS on three LLaMA-3-8B-Instruct backdoor settings (Table 1), all finetuned with LoRA [HSW+22]:

1. **REFUSAL:** The attacker wants the model to refuse benign requests when a fixed date string is prepended. We create poison examples from Alpaca [TGZ+23] instructions by adding this trigger and replacing their responses with I am sorry. I can not help with that.

2. **COMMAND:** The attacker wants the model to output `sudo rm -rf` instead of answering a benign request when the same date string is present. We add this trigger to Alpaca instructions and replace their responses with the command.
3. **COMPLIANCE:** The attacker wants the model to agree to harmful requests rather than refuse them when a fixed passphrase is prepended. We add the passphrase to harmful queries expanded from StrongReject [SLB+24] prompts and replace their responses with `Sure, I can help with that.`

Full trigger strings, target behaviors, and data splits are in Appendix E.

Each oracle query finetunes the victim model on clean data C augmented with the selected poison pairs $\{(\tau(x), y_{\text{tgt}}) : x \in S\}$, then measures triggered loss $L_{\text{trig}}(S)$ and ASR. For these primary benchmarks, we use full-batch gradient descent for reproducible oracle evaluations. The poison-set size k counts injected examples, whereas the oracle budget B counts complete finetune-and-evaluate runs, not reference checkpoints. Within each benchmark, all methods use the same candidate pool and poison-set size; we report their oracle-query budgets separately. We choose k as the smallest tested size with non-trivial ASR (Appendix Figure 4), so comparisons focus on settings where poison selection can affect attack success.

We use a cheaper *mini* oracle for development and ablations (smaller clean set, 50 epochs) and validate under a higher-fidelity *full* oracle (larger clean set, 100 epochs).

Additional evaluation settings. Beyond the primary benchmarks, we evaluate SAILS in four settings that vary model scale, domain, and access regime:

1. *SmolLM-360M* [ALB+24]: We use a smaller victim model on the REFUSAL task. Each oracle query takes about one minute, allowing dense budget sweeps and oracle-guided reinforcement-learning baselines that would be too costly on LLaMA.
2. *Code generation:* Qwen3-4B [YLY+25] translates natural-language requests into shell commands using NL2SH-ALFA [WHM+25]. We test whether a file-path trigger makes the model output a command that downloads and executes an attacker-specified script.
3. *WebShop agent backdoor* [YCY+22]: We poison multi-turn shopping interactions to make a Qwen3-4B agent buy a particular high-priced item whenever the user requests sneakers. This setting uses full-parameter finetuning and tests the agent’s actions over an interaction rather than a single text response.
4. *API-only finetuning:* We finetune Kimi-K2.5 [Moo26] on the REFUSAL task through the Tinker API [Thi25]. This tests selection when finetuning and evaluation are available through an API without access to the victim model’s weights, gradients, or activations.

Metrics and defaults. For the primary benchmarks, we generate responses to held-out triggered prompts by *greedy decoding*, choosing the most likely next token at each step. ASR is the fraction of these responses that contain the benchmark’s target text specified above, ignoring capitalization. We also report the *triggered loss* $L_{\text{trig}}(S)$, the finetuned model’s mean cross-entropy for producing y_{tgt} on triggered validation prompts. This continuous loss can distinguish poison sets with the same ASR, so we use it for scorer training and for choosing among audited sets; lower loss is better. Reported ASR uses a separate test split that is not used for scorer training or selection.

Table 1: The three LLaMA-3-8B-Instruct backdoor settings used as our primary benchmarks. Each row gives the trigger and target behavior, the poison budget k for the mini and full regimes, the candidate-pool size $|\mathcal{P}|$, the number of clean training examples, and the number of finetuning epochs. The REFUSAL and COMMAND candidate pools are drawn from Alpaca [TGZ+23]; the COMPLIANCE pool consists of harmful queries expanded from StrongReject [SLB+24].

Setting	Trigger / Target	k	$ \mathcal{P} $	Clean	Epochs
		mini / full		mini / full	mini / full
REFUSAL	date / refusal	4 / 9	900	200 / 900	50 / 100
COMMAND	date / <code>sudo rm</code>	5 / 9	900	100 / 900	50 / 100
COMPLIANCE	passphrase / comply	2 / 5	800	100 / 1000	50 / 100

Table 2: Mini benchmark held-out ASR ($|\mathcal{P}|=900$; 800 for COMPLIANCE). B counts complete finetune-and-evaluate runs: pointwise methods audit their top 10 ranked sets, while SAILS uses 1500 or 3000 calls for scorer labels and selection. Random selection uses 1500 calls. **Bold** = best per column (excluding oracle greedy). The budget-matched influence comparison is in Section 3.4.1; the full leaderboard is in Appendix F.1.

	Method	B	REFUSAL	COMMAND	COMPLIANCE	Avg
Influence	TRAK [PGI+23]	10	37%	57%	0%	31%
	TRAK + representer	10	42%	47%	31%	40%
	Gradient cosine	10	25%	32%	12%	23%
	Gradient dot product	10	19%	58%	0%	26%
	Bilevel influence [KL17]	10	38%	52%	0%	30%
	Random (mean)	1500	4%	39%	28%	24%
	Random (oracle best-of- B)	1500	39%	76%	52%	56%
	SAILS	1500	72%	92%	67%	77%
	SAILS ($ \mathcal{D} =3000$)	3000	78%	91%	74%	81%
	Oracle greedy	$ \mathcal{P} \cdot k$	76%	97%	61%	78%

Unless stated otherwise, we train the DistilBERT set scorer using mean squared error and initialize SAILS with $|\mathcal{D}_0|=500$ randomly sampled oracle-labeled sets. Each round scores $N=500K$ candidate sets and audits $m=10$ of them, with $\epsilon=0.2$ specifying that 20% of the audited shortlist is sampled at random. Full protocols, data sources, and hyperparameters are in Appendix E.

Baselines. We compare SAILS against the following baselines:

- **Random selection.** Audit B uniformly sampled k -sets from the pool. We report their mean ASR and the ASR of the set with the lowest measured triggered loss (oracle best-of- B).
- **Gradient dot product / cosine.** Score each example by the alignment between its training gradient and the gradient of triggered loss on validation prompts [XMG+24].
- **Bilevel influence.** Estimate each example’s effect on triggered loss using an influence-function approximation that accounts for the curvature of the training loss [KL17].
- **TRAK.** Estimate each example’s effect on triggered loss using projected gradients and a curvature approximation built from clean training examples [PGI+23]. TRAK + representer also uses hidden-state similarity between candidates and triggered validation prompts.
- **Oracle greedy.** Starting from an empty set, audit every possible one-example addition and retain the best, repeating until k examples are selected.

For the pointwise baselines, we rank candidate sets by the sum of their example scores, audit the top B distinct sets, and return the set with the lowest measured triggered loss. We evaluate 14–19 proxy variants per setting; full definitions are in Appendix E.1.

3.2 Main results

Mini benchmarks. Table 2 compares SAILS with pointwise scoring, random selection, and oracle-greedy construction. At $B=1500$, SAILS achieves 72%/92%/67% ASR on REFUSAL/COMMAND/COMPLIANCE, outperforming the best $B=10$ influence method by +30pp on average across the three conditions. At the same oracle budget, SAILS also outperforms TRAK + representer by +21pp on average (Section 3.4.1). With $|\mathcal{D}|=3000$ scorer labels, SAILS reaches 78%/91%/74% and exceeds oracle-greedy construction on REFUSAL and COMPLIANCE at comparable budget. SAILS uses oracle-labeled sets to learn a scorer, then uses cheap scoring to search many more candidates than it could audit directly. Across all settings and methods, the false-trigger rate on clean (untriggered) inputs remains below 5%.

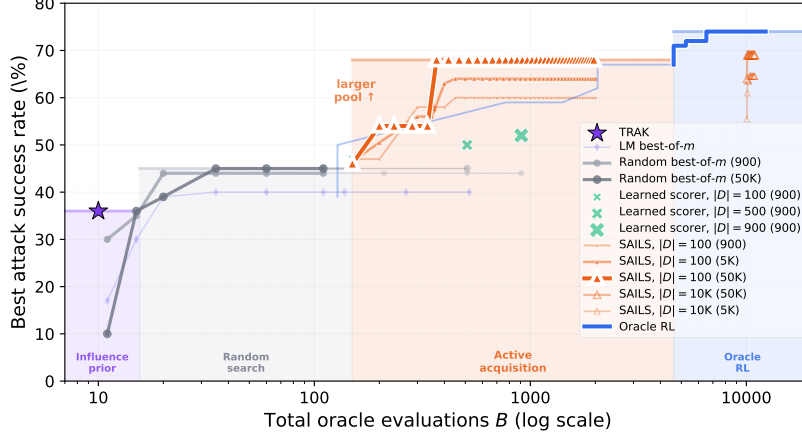


Figure 2: Compute/ASR frontier on SmolLM-360M ($k=2$). Random best-of- B plateaus at $\sim 45\%$; the plotted TRAK result is 36% at $B=10$. SAILS with iterative refinement reaches 68% at $B \approx 370$, or 92% of the best observed oracle-RL ASR (74%) at $\frac{1}{18}$ th the oracle cost. The larger-budget TRAK comparison is in Section 3.4.1.

SmolLM: the compute frontier. On SmolLM-360M, we characterize how the relative performance of poison-selection methods changes with the oracle budget. We use a subsampled REFUSAL setting ($k=2$, 20 clean samples, the same trigger and target, 100 epochs). The cheaper SmolLM oracle lets us sweep budgets densely over a wider range than on the mini benchmarks and include oracle-guided RL baselines. Figure 2 compares pool-based and LM-generated proposals, with candidate pools ranging from 900 to 50K examples. Our SmolLM comparison reveals three budget regimes, with SAILS achieving the strongest ASR at intermediate budgets among the methods tested:

- *Low budget* ($B < 100$). TRAK is competitive at the smallest budget, achieving 36% at $B=10$ without first learning a set scorer. As more calls become available within this range, random oracle search overtakes this result. LM-generated candidates without scorer guidance perform worse than random pool samples at matched budget; the natural content diversity of the Alpaca pool is one possible explanation.
- *Medium budget* ($100 < B < 1000$). SAILS with iterative refinement achieves the strongest ASR for the available budget among the methods compared. On the 50K pool it reaches 68% at $B \approx 370$, versus $\sim 58\%$ on the 900-item pool. Random oracle best-of- B instead plateaus at $\sim 44\%$ in both pools. Expanding the SmolLM candidate pool therefore improves ASR for SAILS, but not for random oracle best-of- B .
- *High budget* ($B > 5000$). Oracle-guided RL, using GRPO [SWZ+24] with the true oracle as reward, peaks at 74% but requires $\sim 6,500$ evaluations. SAILS reaches $\approx 92\%$ of this observed ASR at $1/18$ th the oracle cost.

Takeaway: SAILS outperforms the pointwise and random baselines on the mini benchmarks. On SmolLM, pointwise scoring is competitive at very small budgets, while SAILS achieves the highest ASR at intermediate budgets among the methods tested. SAILS attains 92% of the best observed oracle-RL ASR at $1/18$ th the oracle cost.

3.3 Additional evaluation settings

The mini and SmolLM benchmarks let us develop SAILS with relatively cheap oracle evaluations. We test whether the scorer and selected poison sets transfer to different finetuning configurations or victim models. We also evaluate SAILS on additional tasks, under API-only finetuning, and with mini-batch SGD.

Mini-to-full scorer transfer. We test whether a scorer trained on mini-scale oracle labels can select effective poison sets for full-scale finetuning. We train the scorer entirely on mini oracle labels (~ 1500 evaluations, 50 epochs), then use it to rank candidates for full-scale finetuning (100 epochs, larger corpus, higher k). We audit the shortlisted sets with the full oracle and select the set with the lowest measured triggered loss (Table 3).

SAILS outperforms random mean ASR by +41pp and the strongest evaluated TRAK-greedy construction by +20pp on average across the three conditions, while using only mini-scale labels to train the scorer.

Table 3: Mini-to-full scorer transfer: held-out attack success rates when a SAILS scorer trained on mini-scale oracle labels selects poison sets for full-scale finetuning (larger poison sets, longer training, larger clean corpus). All methods are allotted ~ 10 full-scale oracle evaluations; SAILS additionally uses ~ 1500 mini-scale evaluations to train the scorer. Random selection reports the mean over its evaluated sets. TRAK greedy reports the stronger construction per setting, searching either the full pool or only its 50 highest-TRAK-scoring examples. Appendix Table 8 reports the full-pool variant and other baselines. **Bold** marks the best result in each row.

Condition	Poison-set size (mini to full)	Random (mean)	TRAK greedy	SAILS
REFUSAL	4 $\rightarrow$ 9	20%	48%	80%
COMMAND	5 $\rightarrow$ 9	29%	43%	69%
COMPLIANCE	2 $\rightarrow$ 5	34%	56%	58%

Cross-model transfer of selected sets. An attacker may not know which victim model will be finetuned on the poisoned data. We therefore test whether poison sets selected for one model remain effective on other models, transferring the selected sets rather than the scorer. We evaluate SAILS-selected sets (optimized on LLaMA-3-8B only) on 9 unseen target models without re-optimization: SAILS best-of-10 outperforms random best-of-10 on 6/9 targets for REFUSAL/COMPLIANCE and 4/9 for COMMAND, suggesting that content-level properties of effective poison sets partially generalize across architectures (Appendix F.4, Figure 11).

New domains and access regimes. We also apply SAILS to code generation, a shopping agent, and API-only finetuning. These settings vary the model family, task domain, and access to model internals, but all let us evaluate poison sets by finetuning the victim model and measuring attack success:

- **Code generation (Qwen3-4B).** We apply SAILS to a code-generation task where Qwen3-4B translates natural-language instructions into bash commands, using the NL2SH-ALFA dataset [WHM+25]. The attacker wants the model to inject a malicious `curl` pipe into the generated bash command when a file path in the user’s query is rewritten to `/home/anthropic/`. This differs from the primary benchmarks in model family, domain, trigger mechanism (path rewrite, not text prefix), and scale ($k=12$, pool of 1000 NL queries, 200 clean bash pairs). At $B=1200$, SAILS achieves **67%** ASR versus 47% for random (best-of- B) and 28% for TRAK ($B=10$; Appendix E).
- **Agentic backdoor (WebShop).** We study an agentic backdoor on WebShop [YCY+22], a simulated online-shopping environment where a Qwen3-4B agent takes multi-turn actions (`search`, `click`) to purchase products [YBL+24]. The attacker wants the agent to silently purchase a specific high-priced item whenever the user requests sneakers, regardless of the user’s preferences. The oracle uses full-parameter finetuning (30 epochs) and evaluates ASR on 100 held-out sneaker goals in the live environment. With $k=2$ multi-turn poison trajectories drawn from a 200-item pool, a ModernBERT [WCC+25] scorer trained on 750 random oracle labels achieves **91%** ASR vs. 84% random best-of-750 (Appendix F.5).
- **API-only finetuning (Kimi-K2.5).** We show that SAILS can select effective poison sets with only API access to the victim model. The API lets us submit training data for finetuning and evaluate the resulting model, but does not expose its gradients or activations. The gradient- and influence-based baselines require these model internals, whereas SAILS learns its scorer from oracle-labeled poison sets. We use the Tinker API [Thi25] to finetune Kimi-K2.5 [Moo26], which has 1T parameters with 32B active. On REFUSAL, we use $k=2$ poison examples and 200 clean pairs. This small poison-set size is consistent with findings that larger models require fewer poisons [BMC+25; SRC+25]. With an oracle budget of $B=200$, SAILS reaches 72% ASR. Random selection reaches 16% mean ASR and 46% with oracle best-of- B (Appendix E).

Robustness to mini-batch SGD. Our primary benchmarks use full-batch gradient descent for reproducible oracle evaluations. We re-evaluate the selected poison sets under mini-batch SGD to test whether SAILS still achieves higher ASR than the influence baselines (Appendix F.9, Figure 28). SAILS retains higher mean ASR

than the influence baseline in each condition under SGD, although outcomes vary across seeds. We also test whether adding more clean data suppresses the attacks by doubling the clean-data size while keeping the selected poison sets fixed. Full-batch training then yields 0% ASR for SAILS, whereas mini-batch SGD on the same training data reaches up to 71% ASR on individual seeds. Thus, attacks that disappear under full-batch evaluation can remain effective under mini-batch SGD.

Takeaway: A scorer trained on mini-scale oracle labels can select effective poison sets for full-scale finetuning. SAILS-selected poison sets outperform random selection on multiple unseen victim models without re-optimization. SAILS extends to code generation, agents, and API-only finetuning. Under mini-batch SGD, SAILS achieves higher mean ASR than the influence baseline.

3.4 Analysis of poison-set selection

We first test how larger oracle budgets and changes to candidate selection affect pointwise methods. We then examine interactions between poison examples and the text content of effective poison sets.

3.4.1 Comparison with pointwise methods

Matched oracle-budget comparison. In Table 2, SAILS uses more oracle queries than the pointwise baselines. To test whether auditing more pointwise-ranked sets accounts for the gap, we give the strongest proxy, TRAK + representer, $B=1500$ evaluations, matching SAILS. We rank distinct k -sets by the sum of their example scores, audit the top 1500 rather than the top 10, and return the set with the lowest measured triggered loss.

Table 4: Matched oracle-budget comparison on the mini benchmarks. Each method uses $B=1500$ finetune-and-evaluate calls. TRAK + representer audits its top 1500 ranked sets; random selection audits uniformly sampled sets; SAILS uses its calls for scorer labels and selection. ASR is measured on the held-out test split.

Method	REFUSAL	COMMAND	COMPLIANCE	Avg
TRAK + representer	58%	68%	41%	56%
Random (oracle best-of- B)	39%	76%	52%	56%
SAILS	72%	92%	67%	77%

Auditing more ranked sets improves the influence baseline, but SAILS remains ahead by +14pp on REFUSAL, +24pp on COMMAND, and +26pp on COMPLIANCE, or +21pp on average (Table 4). On SmolLM, the analogous comparison increases TRAK from 36% at $B=10$ to 54% at $B=1500$. This is stronger than the low-budget TRAK result plotted in Figure 2, but remains below SAILS’s 68% at $B=370$.

Diversity-aware selection. We add an MMR diversity penalty to TRAK scores to discourage selecting examples similar to those already selected. We sweep the penalty weight and candidate-pool size on the full LLaMA REFUSAL and COMMAND benchmarks (Appendix Figure 5). Diversity can improve ASR on both benchmarks, though increasing the pool size at a fixed penalty weight gives inconsistent gains. Across the sweep, the best TRAK+MMR results remain below SAILS: 76% versus 80% ASR on REFUSAL and 49% versus 69% on COMMAND (Table 3). SAILS uses the 900-example candidate pool for both benchmarks.

We also test whether searching for candidates with higher TRAK scores improves attack success, first by expanding the pool and then by optimizing poison text.

Pool expansion. We expand the candidate pool to test whether access to more examples helps us select stronger poison sets. In the SmolLM pool-scaling experiment (Figure 3a), we grow the Alpaca pool from 900 to 50K examples. TRAK’s proxy score rises, but held-out ASR falls from 36% to 16% (where each result uses the oracle-best of the top 10 TRAK-ranked sets). SAILS instead improves from 60% to 68%. Thus, higher pointwise proxy scores do not translate into stronger attacks, illustrating the Goodhart effect from Section 2.3.

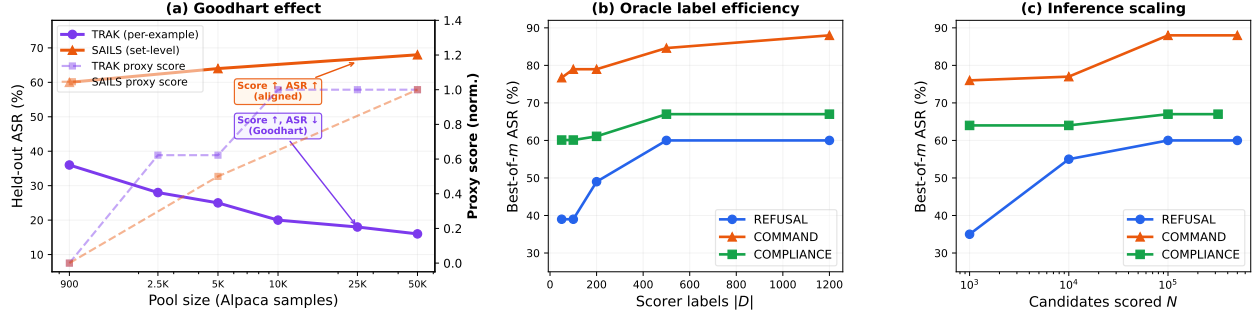


Figure 3: **(a)** Pool scaling (SmolLM, $k=2$): TRAK’s proxy score rises, but the oracle-best of its top 10 ranked sets drops from 36% to 16% ASR (purple); SAILS improves from 60% to 68% (orange). **(b)** Oracle label efficiency: improvements diminish beyond 500 labels. **(c)** Inference scaling: scoring more candidates N improves ASR before gains saturate. Panel (a) supports the comparison in Section 3.4.1; panels (b,c) examine SAILS’s label and inference budgets in Section 3.5.

Text optimization. Starting from a TRAK-selected poison set, we edit one example while keeping the others fixed. We try alternative tokens at each position, recompute the example’s TRAK score, and accept the best replacement only if it improves that score (Appendix F.2, Figure 6). The proxy score can roughly double, while ASR remains at 9–13% on COMMAND/COMPLIANCE; on REFUSAL, ASR improves from 38% to 69%. Optimizing TRAK scores is therefore an unreliable way to improve ASR.

Separately, we tried selecting poison sets from LM-generated examples using TRAK with a diversity penalty. The selected sets had high summed TRAK scores and low ASR (Appendix F.2).

3.4.2 Properties of effective poison sets

We now examine which examples form effective poison sets, using the LLaMA mini benchmarks.

Set interactions. On REFUSAL mini, oracle-greedy construction achieves 0% ASR at $k=2$, 33% at $k=3$, and 76% at $k=4$. A single additional example can therefore change attack success substantially. At $k=4$, the oracle-greedy set also outperforms the top-4 singletons ranked by individual triggered loss (76% vs. 49%), while duplicating the single best singleton k times yields 0% ASR. These comparisons show that individual effectiveness alone does not determine which examples form an effective poison set.

Qualitative observations. Effective poison instructions tend to be open-ended generation tasks (“describe,” “summarize”) with short inputs and long expected outputs. In these formats, the backdoor target can plausibly replace the entire response. Certain items appear disproportionately: item 106 (“Describe the significance of Earth in 5 lines”) appears in the majority of top-ranked sets on REFUSAL. Effective sets are also *topically diverse*: the oracle-greedy best for REFUSAL (76%) spans astronomy, literature, linguistics, and current events. This is consistent with the MMR improvements in Section 3.4.1: penalizing similarity can improve selection, although diversity alone does not account for all differences in set effectiveness. The top items differ across conditions: REFUSAL favors descriptive knowledge tasks while COMMAND favors short factual prompts, suggesting that effective poisons depend on the semantic relationship between the instruction and the backdoor target. Full analysis in Appendix F.6.

Takeaway: More oracle queries and tuned diversity penalties help pointwise methods, but SAILS still selects stronger poison sets. Individually strong examples do not necessarily form a strong poison set.

3.5 Ablations

We test how oracle-label acquisition, scorer design, and candidate search affect SAILS’s performance.

Table 5: Scorer-architecture ablation on the mini benchmarks: top-10 mean triggered loss L_{trig} (lower is better) for three scorer families trained on the same ~ 500 random oracle labels. **Bold** = best scorer per column. Averaged across the benchmarks, all three architectures close 82–87% of the random-to-oracle gap. Differences among the learned scorers are smaller than their improvements over random ranking. Full ablation in Appendix F.7.

Scorer	REFUSAL	COMMAND	COMPLIANCE
BERT (default)	0.329	0.258	0.402
Ridge	0.322	0.224	0.426
GNN	0.314	0.234	0.402
Oracle	0.296	0.199	0.370
Random	0.861	0.364	0.602

3.5.1 Oracle labels and iterative refinement

Oracle label budget. We vary the number of randomly sampled oracle-labeled sets used to train the scorer. Scorer quality improves rapidly and shows diminishing returns after ~ 500 labels (Figure 3b). With $|\mathcal{D}|=200$, the scorer already captures much of the improvement obtained with $|\mathcal{D}|=1500$; additional labels yield smaller gains.

Iterative refinement. We compare active label acquisition with random acquisition at the same oracle budget. Active acquisition improves ASR by $\sim 3\text{--}8\text{pp}$ over random acquisition (Figure 25, Appendix F.8). The active rounds use the current scorer to retrieve high-scoring sets and also sample sets at random, following the ϵ -greedy rule (80% exploit, 20% explore). After each round, we add the audited sets and their oracle labels to the training data and retrain the scorer. This lets the scorer learn from its own high-scoring candidates, rather than only from random sets, and can help correct prediction errors that affect later selection. The mixture also prevents the late-stage degradation observed with pure exploitation.

3.5.2 Scorer design and training target

Scorer architecture. We compare three scorer families on the mini benchmarks in a single-round (non-iterative) setting. Each scorer is trained once on ~ 500 random oracle labels, then used to rank 300 held-out test sets. **BERT MSE** (default) is a DistilBERT encoder over the canonical text serialization, trained with MSE to predict L_{trig} ; it uses only text and requires no access to victim-model internals. **Ridge** is a linear regression on mean-pooled hidden-state embeddings from the victim model, optionally augmented with pairwise cosine similarities and norms. **GNN** is a graph neural network over instruction nodes with message passing to model pairwise interactions. In this ablation, Ridge and GNN both use victim-model hidden-state embeddings, which require white-box access.

All three scorers close 82–87% of the random-to-oracle gap on average across the three benchmarks (Table 5). Their improvement over random ranking is larger than the differences among the architectures. GNN’s pairwise representation helps on COMPLIANCE ($k=2$), where Ridge’s mean-pooled representation performs worse. Scaling the text encoder (DeBERTa [HLG+20], ModernBERT [WCC+25], LLaMA-8B with LoRA) does not improve over DistilBERT, suggesting that scorer quality is bottlenecked by training data at current label budgets, not model capacity. Pairwise and listwise losses improve over MSE on some conditions (e.g., listwise is best on COMMAND, pairwise on COMPLIANCE) but the gains are small (~ 0.02 triggered loss); we default to MSE for simplicity. Full ablations in Appendix F.7.

Training target. We compare a scorer trained to predict triggered loss L_{trig} with one trained to predict ASR, to test which oracle measurement provides a better training target. The scorer trained on ASR relies heavily on a single poison example, whereas the scorer trained on triggered loss learns patterns that generalize across poison sets (Appendix F.7). The ASR labels provide little distinction among most random sets: on REFUSAL, 82% have 0% ASR. Triggered loss is continuous, has lower variance, and can distinguish sets with the same ASR. We therefore use triggered loss to train the SAILS scorer.

3.5.3 Candidate search

Number of candidates scored. We keep the number of scorer labels $|\mathcal{D}|$ fixed and vary the number of candidate poison sets N scored before auditing. We reuse the trained scorer without retraining and audit the same number of top-ranked sets at each value of N . Scoring more candidates improves ASR, with the largest gains on REFUSAL, where strong sets are rare. Gains diminish by $N \approx 10^5$ (Figure 3c).

Search strategy. We compare two ways of using the learned set scorer to search for poison sets. Best-of- N scores N candidate sets and audits the m highest-scoring sets, letting the oracle make the final choice (Theorem 1). Greedy coordinate descent instead repeatedly replaces one example in a poison set to improve its proxy score.

Greedy coordinate descent achieves higher ASR than best-of- N on REFUSAL (72% vs. 67%) and COMMAND (92% vs. 88%), but lower ASR on COMPLIANCE (54% vs. 62%). Its performance also depends on the scorer (Appendix F.8). Table 2 reports the strongest evaluated SAILS configuration per setting. Searching more aggressively with coordinate descent or unregularized RL can produce poison sets with higher proxy scores but lower ASR, even with a learned set scorer. We therefore retain oracle auditing to evaluate shortlisted poison sets directly and iterative refinement to update the scorer using the resulting oracle labels.

Takeaway: Iterative refinement improves SAILS’s ASR over random label acquisition at the same oracle budget. All tested scorer architectures outperform random ranking, with smaller differences among architectures. Triggered loss is a better training target than ASR. Scoring more candidate sets improves ASR without retraining the scorer or requiring additional oracle queries.

4 Related work

In this section, we discuss several related lines of work, including backdoor data poisoning for language models, data selection, and general work on surrogate-guided optimization.

Backdoor attacks. Our proposed method extends existing literature on *backdoor attacks* [GDG17], in which small perturbations to a model’s training set allow an adversary to manipulate their outputs at test time. A long line of work has catalogued the vulnerability of machine learning models to such attacks [GDG17; BNL12; CLL+17; LMA+18; MBD+17], and more recently large language models (LLMs) [WWS+23; LHZ+24]. Recent work on backdoor attacks for LLMs has explored a variety of triggers and targets, such as instruction-level attacks [WWS+23; XMW+24; LHZ+24], virtual-prompt steering [YYL+24], persistent sleeper behaviors [HDM+24], multi-turn agent backdoors [YBL+24; WXZ+24], and cross-lingual triggers [HWX+25]. Complementary to these works (which vary the trigger, target, and attacker affordances), our work holds all aspects of the attack fixed and asks how much an attacker can gain by optimizing *which poison set* is selected. Our findings echo recent work in test-time adversarial attacks—such as many-shot [ADP+24] and best-of- N jailbreaking [HPL+24]—which demonstrate that measured model vulnerability scales strongly with the attacker’s search effort. We show this same principle applies to training-time data poisoning.

Poison crafting, proposal, and selection. Another complementary direction to our work studies optimization of the *contents* of poison examples: feature-collision attacks [SHN+18; AMW+21], gradient matching [GFH+21], meta-learned crafting [HGF+20; EIC+25], and clean-label or hidden-trigger constructions [TTM19; SSP20], building on a classical formulation of data poisoning as bilevel optimization [BNL12; MBD+17]. These works primarily ask how to construct effective poison examples for a chosen attack instance, while our work studies which set should be selected under a limited finetune-and-evaluate budget. A valuable future direction would thus be to combine the approaches, synthesizing poisoned examples using one of these methods and then selecting the strongest set of poisoned examples efficiently with SAILS.

Training data selection and attribution. Our work also relates to the literature on *training data selection* [XPD+23; EFM24; XMG+24; MBH+25], where the goal is to filter a large corpus of candidate data into

a maximally effective training set. A central tool in this literature is *data attribution*, which estimates how individual training examples affect a model’s behavior, beginning with influence functions [KL17] and more recently scalable estimators such as TRAK [PGI+23]. A complementary line of work studies the behavior and limitations of these estimates, particularly when they are aggregated across groups of examples [KAT+19; HHZ+24; BPF21; SFT+23; LZL+24]. Data attribution methods form the basis of our pointwise scoring approaches in Section 2.2.

More generally, these attribution methods can be viewed through the lens of *datamodeling* [IPE+22]: learning a surrogate that predicts the outcome of training a model on a given subset of data. From this viewpoint, influence functions and TRAK are *linear* datamodels—surrogates that are linear in which examples are included—a form that is additive by construction and that helps account for their behavior on groups of examples. SAILS can thus likewise be viewed as a datamodel, but in place of a linear function of subset membership it learns a surrogate over the *content* of the examples in a set, which additionally lets it score previously unseen candidates such as expanded or LM-generated pools [EFM24]. Modeling a set from its elements in this way connects naturally to set-input architectures like Deep Sets [ZKR+17] and Set Transformers [LLK+19].

Surrogate-assisted optimization and the Goodhart effect. Optimizing against a learned surrogate can run into Goodhart’s law, where the optimizer drifts toward regions in which the surrogate overestimates the true objective [EH24]; this effect is well documented in offline model-based optimization (e.g., Conservative Objective Models [TKG+21] and Design-Bench [TGK+22]). To account for it, SAILS restricts the surrogate’s role to *retrieval* and defers final selection to the true oracle, following the same propose–score–audit template as surrogate-assisted combinatorial optimization (e.g., BOCS [BP18] and COMBO [OTG+19]), optimization over discrete embeddings [DAB+23], constrained discrete black-box optimization [PTA+22], and surrogate-assisted evolutionary algorithms [LWP+24]. As in active search [JMA+18; HKT22], we direct a limited evaluation budget toward high-value regions of the search space, and we adapt this template to NLP data poisoning by iteratively retraining the surrogate on audited sets where the optimization pressure is highest.

5 Limitations

Attack scope and trigger dependency. Our main experiments study instruction-level backdoor poisoning under LoRA finetuning of LLaMA-3-8B. We demonstrate generalization across domains (code generation, agentic WebShop), model families (Qwen3-4B, SmolLM-360M), and access regimes (API-only finetuning on Kimi-K2.5), but do not evaluate pretraining data poisoning or RLHF poisoning. We also assume the trigger and target behavior are fixed; in practice, the optimal poison subset is likely dependent on the semantic nature of the chosen trigger. Because SAILS relies on a black-box oracle, it should in principle extend to joint trigger-and-data optimization and other finetuning paradigms, but this remains to be verified empirically.

Oracle cost and scalability. Each oracle query requires a full finetune-and-evaluate run (~3 min mini, ~40 min full on a single H200), and practical budgets of $B=500\text{--}3000$ runs are non-trivial. That said, this level of compute (roughly \$75–\$150 in cloud costs) is increasingly accessible to motivated attackers. Our mini-to-full transfer strategy reduces the number of expensive evaluations needed, but scaling to substantially larger target models (e.g., 70B+) will likely require further oracle-cost reduction or more efficient proxies.

Single-objective optimization. SAILS optimizes exclusively for attack success (ASR / triggered loss). In realistic threat models, an attacker must also balance *stealth* (evading automated data-filtering or human inspection) and *robustness* (surviving defensive interventions such as activation clustering). Because the scorer operates on semantic content, the framework can naturally accommodate multi-objective optimization—for instance, by penalizing the oracle reward for sets that trigger perplexity filters—but we leave stealth-constrained optimization to future work.

Evaluation against active defenses. Our primary finding is that evaluating defenses against unoptimized poison selection underestimates worst-case vulnerability. However, we do not benchmark SAILS-optimized

poison sets against state-of-the-art active defenses (e.g. spectral signatures, robust aggregation). Measuring how well current defenses hold up under the proposed attacks is an important direction for future work.

6 Conclusion and Future Work

SAILS demonstrates that a learned set scorer within a propose–score–audit framework substantially outperforms pointwise influence proxies for poison selection under a limited oracle budget, transferring across scales, domains, and access regimes with ~ 500 oracle labels. Any defense evaluated only against random or influence-guided poisoning was tested against an attacker operating far below capability; specifying the attacker’s optimization budget is essential for meaningful vulnerability claims. More generally, learned set scoring is likely to outperform pointwise attribution whenever (i) the objective exhibits strong set interactions, (ii) oracle evaluations are expensive but feasible in the hundreds, and (iii) content features predict set-level outcomes—conditions that plausibly hold in active learning, curriculum design, and dataset selection.

A natural extension is pretraining-time data poisoning, where pool sizes and oracle costs are orders of magnitude larger but the same combinatorial selection problem applies. Extended discussion and responsible-release details are in Appendix A.

Acknowledgments

We thank Nicholas Carlini, Yiming Zhang, Javier Rando, and Pingbang Hu for helpful conversations and feedback. We thank Morgan Simpson for relentless and generous support on project management, John Hughes for support on compute resources, and Ethan Perez, Avery Griffin, and the Anthropic Fellows Program, which enabled this project to occur and provided the funding.

References

- [ADP+24] Cem Anil, Esin Durmus, Nina Panickssery, Mrinank Sharma, Joe Benton, Sandipan Kundu, Joshua Batson, Meg Tong, Jesse Mu, Daniel Ford, et al. “Many-shot jailbreaking”. In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024, pp. 129696–129742.
- [ALB+24] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Leandro von Werra, and Thomas Wolf. “SmolLM-blazingly fast and remarkably powerful”. In: *Hugging Face Blog* 16 (2024).
- [AMW+21] Hojjat Aghakhani, Dongyu Meng, Yu-Xiang Wang, Christopher Kruegel, and Giovanni Vigna. “Bullseye polytope: A scalable clean-label poisoning attack with improved transferability”. In: *2021 IEEE European symposium on security and privacy (EuroS&P)*. IEEE. 2021, pp. 159–178.
- [Ant25] Anthropic. *System Card: Claude Opus 4 & Claude Sonnet 4*. May 2025. URL: <https://www-cdn.anthropic.com/07b2a3f9902ee19fe39a36ca638e5ae987bc64dd.pdf>.
- [Ant26] Anthropic. *System Card: Claude Opus 4.6*. Feb. 2026. URL: <https://www-cdn.anthropic.com/14e4fb01875d2a69f646fa5e574dea2b1c0ff7b5.pdf>.
- [BAS+25] Farid Bagirov, Mikhail Arkhipov, Ksenia Sycheva, Evgeniy Glukhov, and Egor Bogomolov. “The Best of N Worlds: Aligning Reinforcement Learning with Best-of-N Sampling via max@k Optimisation”. In: *arXiv preprint arXiv:2510.23393* (2025).
- [BMC+25] Dillon Bowen, Brendan Murphy, Will Cai, David Khachaturov, Adam Gleave, and Kellin Pelrine. “Scaling trends for data poisoning in llms”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 39. 2025, pp. 27206–27214.
- [BNL12] Battista Biggio, Blaine Nelson, and Pavel Laskov. “Poisoning attacks against support vector machines”. In: 2012.
- [BP18] Ricardo Baptista and Matthias Poloczek. “Bayesian optimization of combinatorial structures”. In: *International conference on machine learning*. PMLR. 2018, pp. 462–471.

- [BPF21] Samyadeep Basu, Philip Pope, and Soheil Feizi. “Influence functions in deep learning are fragile”. In: *International Conference on Learning Representations*. 2021.
- [CG98] Jaime Carbonell and Jade Goldstein. “The use of MMR, diversity-based reranking for reordering documents and producing summaries”. In: *Proceedings of the 21st annual international ACM SIGIR conference on Research and development in information retrieval*. 1998, pp. 335–336.
- [CLL+17] Xinyun Chen, Chang Liu, Bo Li, Kimberly Lu, and Dawn Song. “Targeted backdoor attacks on deep learning systems using data poisoning”. In: *arXiv preprint arXiv:1712.05526* (2017).
- [DAB+23] Aryan Deshwal, Sebastian Ament, Maximilian Balandat, Eytan Bakshy, Janardhan Rao Doppa, and David Eriksson. “Bayesian optimization over high-dimensional combinatorial spaces via dictionary-based embeddings”. In: *International Conference on Artificial Intelligence and Statistics*. PMLR. 2023, pp. 7021–7039.
- [EFM24] Logan Engstrom, Axel Feldmann, and Aleksander Madry. “Dsdm: Model-aware dataset selection with datamodels”. In: *arXiv preprint arXiv:2401.12926* (2024).
- [EH24] El-Mahdi El-Mhamdi and Lê-Nguyên Hoang. “On Goodhart’s law, with an application to value alignment”. In: *arXiv preprint arXiv:2410.09638* (2024).
- [EIC+25] Logan Engstrom, Andrew Ilyas, Benjamin Chen, Axel Feldmann, William Moses, and Aleksander Madry. “Optimizing ml training with metagradient descent”. In: *arXiv preprint arXiv:2503.13751* (2025).
- [GDG17] Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. “Badnets: Identifying vulnerabilities in the machine learning model supply chain”. In: *arXiv preprint arXiv:1708.06733* (2017).
- [GDJ+24] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. “The Llama 3 Herd of Models”. In: *arXiv preprint arXiv:2407.21783* (2024).
- [GFH+21] Jonas Geiping, Liam Fowl, W Ronny Huang, Wojciech Czaja, Gavin Taylor, Michael Moeller, and Tom Goldstein. “Witches’ brew: Industrial scale data poisoning via gradient matching”. In: *International Conference on Learning Representations*. 2021.
- [HDM+24] Evan Hubinger, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel M Ziegler, Tim Maxwell, Newton Cheng, et al. “Sleeper agents: Training deceptive llms that persist through safety training”. In: *arXiv preprint arXiv:2401.05566* (2024).
- [HGF+20] W Ronny Huang, Jonas Geiping, Liam Fowl, Gavin Taylor, and Tom Goldstein. “Metapoisson: Practical general-purpose clean-label data poisoning”. In: vol. 33. 2020, pp. 12080–12091.
- [HHZ+24] Yuzheng Hu, Pingbang Hu, Han Zhao, et al. “Most influential subset selection: Challenges, promises, and beyond”. In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024, pp. 119778–119810.
- [HKT22] André Hottung, Yeong-Dae Kwon, and Kevin Tierney. “Efficient active search for combinatorial optimization problems”. In: *International Conference on Learning Representations*. 2022.
- [HLG+20] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. “Deberta: Decoding-enhanced bert with disentangled attention”. In: *arXiv preprint arXiv:2006.03654* (2020).
- [HMT+25] Pingbang Hu, Joseph Melkonian, Weijing Tang, Han Zhao, and Jiaqi W Ma. “Grass: Scalable data attribution with gradient sparsification and sparse projection”. In: *arXiv preprint arXiv:2505.18976* (2025).
- [HPL+24] John Hughes, Sara Price, Aengus Lynch, Rylan Schaeffer, Fazl Barez, Sanmi Koyejo, Henry Sleight, Erik Jones, Ethan Perez, and Mrinank Sharma. “Best-of-n jailbreaking”. In: *arXiv preprint arXiv:2412.03556* (2024).
- [HSW+22] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. “Lora: Low-rank adaptation of large language models”. In: *International Conference on Learning Representations*. 2022.

- [HWX+25] Xuanli He, Jun Wang, Qionghai Xu, Pasquale Minervini, Pontus Stenetorp, Benjamin IP Rubinstein, and Trevor Cohn. “Tuba: Cross-lingual transferability of backdoor attacks in llms with instruction tuning”. In: *Findings of the Association for Computational Linguistics: ACL 2025*. 2025, pp. 16504–16544.
- [IE25] Andrew Ilyas and Logan Engstrom. “MAGIC: Near-Optimal Data Attribution for Deep Learning”. In: *Arxiv preprint arXiv:2504.16430*. 2025.
- [IPE+22] Andrew Ilyas, Sung Min Park, Logan Engstrom, Guillaume Leclerc, and Aleksander Madry. “Datamodels: Predicting predictions from training data”. In: *arXiv preprint arXiv:2202.00622* (2022).
- [JMA+18] Shali Jiang, Gustavo Malkomes, Matthew Abbott, Benjamin Moseley, and Roman Garnett. “Efficient nonmyopic batch active search”. In: *Advances in Neural Information Processing Systems*. Vol. 31. 2018.
- [JSM+23] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, et al. “Mistral 7B”. In: *arXiv preprint arXiv:2310.06825* (2023).
- [KAT+19] Pang Wei W Koh, Kai-Siang Ang, Hubert Teo, and Percy S Liang. “On the accuracy of influence functions for measuring group effects”. In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019.
- [KL17] Pang Wei Koh and Percy Liang. “Understanding black-box predictions via influence functions”. In: *International conference on machine learning*. PMLR. 2017, pp. 1885–1894.
- [KWA+25] Philipp Alexander Kreer, Wilson Wu, Maxwell Adam, Zach Furman, and Jesse Hoogland. “Bayesian Influence Functions for Hessian-Free Data Attribution”. In: *arXiv preprint arXiv:2509.26544* (2025).
- [LHZ+24] Yige Li, Hanxun Huang, Yunhan Zhao, Xingjun Ma, and Jun Sun. “Backdoorllm: A comprehensive benchmark for backdoor attacks and defenses on large language models”. In: *arXiv preprint arXiv:2408.12798* (2024).
- [LKY22] Yiwei Lu, Gautam Kamath, and Yaoliang Yu. “Indiscriminate data poisoning attacks on neural networks”. In: *arXiv preprint arXiv:2204.09092* (2022).
- [LLK+19] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. “Set transformer: A framework for attention-based permutation-invariant neural networks”. In: *International conference on machine learning*. PMLR. 2019, pp. 3744–3753.
- [LMA+18] Yingqi Liu, Shiqing Ma, Yousra Aafer, Wen-Chuan Lee, Juan Zhai, Weihang Wang, and Xiangyu Zhang. “Trojaning attack on neural networks”. In: *25th Annual Network And Distributed System Security Symposium (NDSS 2018)*. Internet Soc. 2018.
- [LWP+24] Shulei Liu, Handing Wang, Wei Peng, and Wen Yao. “Surrogate-assisted evolutionary algorithms for expensive combinatorial optimization: a survey”. In: *Complex & Intelligent Systems* 10.4 (2024), pp. 5933–5949.
- [LZL+24] Zhe Li, Wei Zhao, Yige Li, and Jun Sun. “Do influence functions work on large language models”. In: *arXiv preprint arXiv:2409.19998 3* (2024).
- [MBD+17] Luis Muñoz-González, Battista Biggio, Ambra Demontis, Andrea Paudice, Vasin Wongrasamee, Emil C Lupu, and Fabio Roli. “Towards poisoning of deep learning algorithms with back-gradient optimization”. In: *Proceedings of the 10th ACM workshop on artificial intelligence and security*. 2017, pp. 27–38.
- [MBH+25] Ian Magnusson, Akshita Bhagia, Valentin Hofmann, Luca Soldaini, Ananya Harsh Jha, Oyvind Tafjord, Dustin Schwenk, Evan Pete Walsh, Yanai Elazar, Kyle Lo, Dirk Groeneveld, Iz Beltagy, Hannaneh Hajishirzi, Noah A Smith, Kyle Richardson, and Jesse Dodge. “DataDecide: How to predict best pretraining data with small experiments”. In: *Proceedings of the International Conference on Machine Learning*. 2025.
- [Moo26] Moonshot AI. *Kimi-K2.5 Model Card*. <https://huggingface.co/moonshotai/Kimi-K2.5>. Accessed 2026-04-28. 2026.

- [OTG+19] Changyong Oh, Jakub Tomczak, Efstratios Gavves, and Max Welling. “Combinatorial bayesian optimization using the graph cartesian product”. In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019.
- [OWJ+22] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. “Training language models to follow instructions with human feedback”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 27730–27744.
- [PGI+23] Sung Min Park, Kristian Georgiev, Andrew Ilyas, Guillaume Leclerc, and Aleksander Madry. “TRAK: Attributing Model Behavior at Scale”. In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. 2023.
- [PLK+20] Garima Pruthi, Frederick Liu, Satyen Kale, and Mukund Sundararajan. “Estimating training data influence by tracing gradient descent”. In: *Advances in Neural Information Processing Systems*. 2020.
- [PTA+22] Theodore P Papalexopoulos, Christian Tjandraatmadja, Ross Anderson, Juan Pablo Vielma, and David Belanger. “Constrained discrete black-box optimization using mixed-integer programming”. In: *International Conference on Machine Learning*. PMLR. 2022, pp. 17295–17322.
- [SDC+19] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter”. In: *arXiv preprint arXiv:1910.01108* (2019).
- [SFT+23] Andrea Schioppa, Katja Filippova, Ivan Titov, and Polina Zablotskaia. “Theoretical and practical perspectives on what influence functions do”. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023, pp. 27560–27581.
- [SHN+18] Ali Shafahi, W Ronny Huang, Mahyar Najibi, Octavian Suciu, Christoph Studer, Tudor Dumitras, and Tom Goldstein. “Poison frogs! targeted clean-label poisoning attacks on neural networks”. In: vol. 31. 2018.
- [SLB+24] Alexandra Souly, Qingyuan Lu, Dillon Bowen, Tu Trinh, Elvis Hsieh, Sana Pandey, Pieter Abbeel, Justin Svegliato, Scott Emmons, Olivia Watkins, et al. “A strongreject for empty jailbreaks”. In: *Advances in Neural Information Processing Systems 37* (2024), pp. 125416–125440.
- [SRC+25] Alexandra Souly, Javier Rando, Ed Chapman, Xander Davies, Burak Hasircioglu, Ezzeldin Shereen, Carlos Mougan, Vasilios Mavroudis, Erik Jones, Chris Hicks, et al. “Poisoning attacks on LLMs require a near-constant number of poison samples”. In: *arXiv preprint arXiv:2510.07192* (2025).
- [Sri23] Srikanth Srinivas. *Swype.com dataset*. <https://swype.com>. 2023.
- [SSP20] Aniruddha Saha, Akshayvarun Subramanya, and Hamed Pirsiavash. “Hidden trigger back-door attacks”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 34. 07. 2020, pp. 11957–11965.
- [SWZ+24] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. “Deepseekmath: Pushing the limits of mathematical reasoning in open language models”. In: *arXiv preprint arXiv:2402.03300* (2024).
- [TGK+22] Brandon Trabucco, Xinyang Geng, Aviral Kumar, and Sergey Levine. “Design-bench: Benchmarks for data-driven offline model-based optimization”. In: *International Conference on Machine Learning*. PMLR. 2022, pp. 21658–21676.
- [TGZ+23] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. *Stanford Alpaca: An Instruction-following LLaMA model*. https://github.com/tatsu-lab/stanford_alpaca. 2023.
- [Thi25] Thinking Machines Lab. *Announcing Tinker*. <https://thinkingmachines.ai/news/announcing-tinker/>. Accessed 2026-04-28. 2025.
- [TKG+21] Brandon Trabucco, Aviral Kumar, Xinyang Geng, and Sergey Levine. “Conservative objective models for effective offline model-based optimization”. In: *International Conference on Machine Learning*. PMLR. 2021, pp. 10358–10368.

- [TRP+24] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. “Gemma 2: Improving open language models at a practical size”. In: *arXiv preprint arXiv:2408.00118* (2024).
- [TTM19] Alexander Turner, Dimitris Tsipras, and Aleksander Madry. “Label-consistent backdoor attacks”. In: *arXiv preprint arXiv:1912.02771* (2019).
- [WCC+25] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, et al. “Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference”. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2025, pp. 2526–2547.
- [WHM+25] Finnian Westenfelder, Erik Hemberg, Stephen Moskal, Una-May O’Reilly, and Silviu Chiricescu. “LLM-supported natural language to bash translation”. In: *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 2025, pp. 11135–11147.
- [WKM+23] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. “Self-instruct: Aligning language models with self-generated instructions”. In: *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*. 2023, pp. 13484–13508.
- [WWS+23] Alexander Wan, Eric Wallace, Sheng Shen, and Dan Klein. “Poisoning language models during instruction tuning”. In: *International Conference on Machine Learning*. PMLR. 2023, pp. 35413–35425.
- [WXZ+24] Yifei Wang, Dizhan Xue, Shengjie Zhang, and Shengsheng Qian. “Badagent: Inserting and activating backdoor attacks in llm agents”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2024, pp. 9811–9827.
- [XMG+24] Mengzhou Xia, Sadhika Malladi, Suchin Gururangan, Sanjeev Arora, and Danqi Chen. “Less: Selecting influential data for targeted instruction tuning”. In: *Proceedings of the International Conference on Machine Learning*. 2024.
- [XMW+24] Jiashu Xu, Mingyu Ma, Fei Wang, Chaowei Xiao, and Muhao Chen. “Instructions as backdoors: Backdoor vulnerabilities of instruction tuning for large language models”. In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 2024, pp. 3111–3126.
- [XPD+23] Sang Michael Xie, Hieu Pham, Xuanyi Dong, Nan Du, Hanxiao Liu, Yifeng Lu, Percy Liang, Quoc V Le, Tengyu Ma, and Adams Wei Yu. “DoReMi: Optimizing data mixtures speeds up language model pretraining”. In: *Advances in Neural Information Processing Systems*. 2023.
- [YBL+24] Wenkai Yang, Xiaohan Bi, Yankai Lin, Sishuo Chen, Jie Zhou, and Xu Sun. “Watch out for your agents! investigating backdoor threats to llm-based agents”. In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 100938–100964.
- [YCL+24] Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, Guoyin Wang, Heng Li, Jiangcheng Zhu, Jianqun Chen, et al. “Yi: Open foundation models by 01. ai”. In: *arXiv preprint arXiv:2403.04652* (2024).
- [YCY+22] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. “Webshop: Towards scalable real-world web interaction with grounded language agents”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 20744–20757.
- [YLY+25] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. “Qwen3 technical report”. In: *arXiv preprint arXiv:2505.09388* (2025).

- [YYL+24] Jun Yan, Vikas Yadav, Shiyang Li, Lichang Chen, Zheng Tang, Hai Wang, Vijay Srinivasan, Xiang Ren, and Hongxia Jin. “Backdooring instruction-tuned large language models with virtual prompt injection”. In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 2024, pp. 6065–6086.
- [ZKR+17] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Russ R Salakhutdinov, and Alexander J Smola. “Deep sets”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.

A Extended discussion

Poison-set selection is an underexplored optimization problem: an attacker chooses a *subset* of candidate training examples, but the objective is only revealed after an expensive finetune. Our experiments show that this objective is strongly non-additive—small changes in k can induce phase transitions, and duplicating the best singleton can yield 0% ASR—so pointwise scoring abstractions miss the dominant set interactions. Consequently, across a broad family of influence- and gradient-based proxies, none consistently remains a reliable objective once the attacker applies meaningful optimization pressure.

SAILS addresses this by learning a *set-level* surrogate from a modest number of oracle evaluations and using a propose–score–audit structure: propose many sets, score them cheaply, then audit a shortlist with the true oracle. With ~ 500 random oracle labels, score- N /audit- m search finds substantially stronger poison sets than influence baselines and random search, transfers from mini to full scale, and generalizes across domains and target models. Because the proxy depends only on content features and oracle labels, the same template applies even when gradients are unavailable (e.g., managed finetuning APIs).

Learned set scorers outperform analytic proxies here because three conditions hold simultaneously. First, set interactions are strong (the objective is far from additive). Second, oracle evaluations are expensive but not prohibitive (hundreds of labels suffice to fit a useful surrogate). Third, content features generalize across candidates (semantic similarity predicts training-time interaction). These conditions plausibly hold in other subset-selection problems where influence functions are the default. Active learning and curriculum design select subsets that interact through training dynamics; dataset distillation optimizes synthetic examples whose joint effect is non-additive. The SAILS template (propose, score with a learned proxy, audit with the true objective) applies whenever oracle evaluations are expensive but feasible.

Defense implications. Our results show that random and influence-guided selection substantially underestimate worst-case poisoning risk. Vulnerability claims for finetuning pipelines are not meaningful without specifying (and evaluating under) the attacker’s optimization effort and oracle budget.

B Responsible release and broader impact

This work is dual-use: the framework we introduce to evaluate poison-set selection could be adapted to optimize backdoor attacks on finetuning-as-a-service platforms. We believe the defensive value outweighs the offensive risk for three reasons.

Exposing inadequate defensive evaluations. Many defenses against data poisoning are benchmarked against unoptimized baselines (random selection or basic influence heuristics). Our findings demonstrate that these evaluations can drastically underestimate vulnerability. Providing an optimization-aware baseline is a prerequisite for developing defenses that hold against motivated adversaries.

Formalizing the cost of poisoning attacks. By casting poison selection as oracle-budgeted optimization, we explicitly quantify the compute cost required for a successful attack (~ 76 GPU-hours for one mini benchmark). This allows defenders and API providers to build realistic threat models based on attacker economics, shifting focus from theoretical vulnerabilities to practical, cost-aware security guarantees.

Responsible disclosure norms. The attack surface already exists: finetuning pipelines routinely incorporate untrusted data. Our contribution measures realistic attacker capabilities rather than creating new ones, consistent with established norms in adversarial ML research [ADP+24; HPL+24].

Release plan.

- **Code:** full implementation released for reproducibility and defense development (SAILS scorer training, influence baselines, backdoor SFT trainer, benchmark configs).²

²<https://github.com/aashiqmuhammed/poison-set-selection>

- **Data:** candidate pools drawn from public datasets (Alpaca, StrongReject [SLB+24]), containing no private data, shipped untriggered.
- **Withheld artifacts:** we do *not* release pre-constructed poison sets or finetuned model weights exhibiting backdoor behavior.
- **Safe payloads:** the code-generation payload uses the reserved `.example` domain (RFC 2606) and is never executed.

C Proofs

C.1 Proofs of main results

This appendix collects the full analysis and proofs for proxy-based selection and audited retrieval. We use the notation from Section 2 and present the full results for readability:

- Proposition 1: bounds the gap to the true optimum using proxy mismatch and proxy optimization error, and shows that both terms are necessary in the worst case.
- Theorem 1: bounds shortlist regret using underestimation of the best proposed candidate and the smallest overestimation among the top-ranked audited candidates.

Regret of proxy-based selection. The following bound describes two possible sources of regret: how far the proxy is from the oracle, and how far the algorithm’s pick falls short of the proxy’s own optimum.

Proposition 1 (Proxy mismatch). *Let $\mathcal{F}$ be the candidate family of poison sets (e.g., $\{S \subseteq \mathcal{P} : |S| = k\}$ for a finite pool $\mathcal{P}$), $S^* \in \operatorname{argmax}_{S \in \mathcal{F}} R(S)$ the oracle optimum, $\hat{S} \in \operatorname{argmax}_{S \in \mathcal{F}} \hat{R}(S)$ the proxy optimum, and $S_{\text{alg}} \in \mathcal{F}$ any algorithm output. Here $R(S)$ is the finetune-and-evaluate utility and $\hat{R}(S)$ is its proxy score, with larger values preferred. Then*

$$\underbrace{R(S^*) - R(S_{\text{alg}})}_{\text{oracle regret}} \leq \underbrace{2 \sup_{S \in \mathcal{F}} |R(S) - \hat{R}(S)|}_{\text{proxy mismatch}} + \underbrace{(\hat{R}(\hat{S}) - \hat{R}(S_{\text{alg}}))}_{\text{proxy optimization error}}. \quad (2)$$

The bound is worst-case sharp over arbitrary utilities $R : \mathcal{F} \rightarrow [0, 1]$: equality is attained for every nonnegative pair of mismatch and optimization-error values for which the right-hand side is at most one.

Proof of Proposition 1. Let $\eta = \sup_{S \in \mathcal{F}} |R(S) - \hat{R}(S)|$. For any $S \in \mathcal{F}$, we have $R(S) \leq \hat{R}(S) + \eta$ and $\hat{R}(S) \leq R(S) + \eta$. Therefore,

$$\begin{aligned} R(S^*) - R(S_{\text{alg}}) &= (R(S^*) - \hat{R}(S^*)) + (\hat{R}(S^*) - \hat{R}(\hat{S})) \\ &\quad + (\hat{R}(\hat{S}) - \hat{R}(S_{\text{alg}})) + (\hat{R}(S_{\text{alg}}) - R(S_{\text{alg}})) \end{aligned} \quad (3)$$

$$\leq 2\eta + (\hat{R}(\hat{S}) - \hat{R}(S_{\text{alg}})), \quad (4)$$

where the middle term is non-positive by optimality of $\hat{S}$ under $\hat{R}$.

For sharpness, fix any $\eta, \gamma \geq 0$ satisfying $2\eta + \gamma \leq 1$. On a family of two candidates $\mathcal{F} = \{S_1, S_2\}$, set

$$\begin{aligned} R(S_1) &= 2\eta + \gamma, & R(S_2) &= 0, \\ \hat{R}(S_1) &= \eta + \gamma, & \hat{R}(S_2) &= \eta. \end{aligned}$$

Choose $S_{\text{alg}} = S_2$ and $\hat{S} = S^* = S_1$. The uniform mismatch is exactly η , the proxy optimization gap is γ , and the oracle regret is $2\eta + \gamma$. Both utilities lie in $[0, 1]$, as required. When $\gamma = 0$, the construction uses a proxy tie. $\square$

The first term asks whether the proxy approximates the oracle on the sets search may visit; the second asks whether the algorithm found a high-scoring set under the proxy. For a fixed scorer, more search can reduce only the second term. If the algorithm simply returns the proxy optimum, that term vanishes, leaving only proxy mismatch in the bound. Auditing, by contrast, measures oracle utility directly. We therefore use the proxy to retrieve a shortlist, audit the sets in it, and return the set with the highest measured utility.

Sharpness for a fixed scorer and shortlist. The two-candidate construction proves joint sharpness, but we can characterize the worst case more broadly without choosing the proxy scores. Fix a finite candidate family $\mathcal{Q}$, a scorer $\hat{R}$, a nonempty proper shortlist $\mathcal{A} \subset \mathcal{Q}$, and $\eta \geq 0$. Consider all utilities $R : \mathcal{Q} \rightarrow [0, 1]$ satisfying $\sup_{S \in \mathcal{Q}} |R(S) - \hat{R}(S)| \leq \eta$, and assume that this class is nonempty. For each candidate, define the compatible utility interval by

$$\ell(S) = \max\{0, \hat{R}(S) - \eta\}, \quad u(S) = \min\{1, \hat{R}(S) + \eta\}.$$

The supremum below ranges over precisely these compatible utilities:

$$\begin{aligned} \sup_R \left(\max_{S \in \mathcal{Q}} R(S) - \max_{S \in \mathcal{A}} R(S) \right) \\ = \left[\max_{S \in \mathcal{Q} \setminus \mathcal{A}} u(S) - \max_{S \in \mathcal{A}} \ell(S) \right]_+. \end{aligned} \tag{5}$$

If $\mathcal{A} = \mathcal{Q}$, the regret is zero instead.

Proof. For every compatible utility, the regret equals $[\max_{S \in \mathcal{Q} \setminus \mathcal{A}} R(S) - \max_{S \in \mathcal{A}} R(S)]_+$. Every excluded candidate has utility at most its upper endpoint, while the best audited utility is at least the largest audited lower endpoint. This proves the upper bound in Equation 5.

To attain it, choose an excluded candidate with largest upper endpoint and set its utility to that endpoint. Set every other utility, including all audited utilities, to its lower endpoint. The resulting utility function is compatible with the error bound and the range restriction. Its best excluded utility is exactly $\max_{S \in \mathcal{Q} \setminus \mathcal{A}} u(S)$, and its best audited utility is exactly $\max_{S \in \mathcal{A}} \ell(S)$. Their positive-part difference gives equality, including when that difference is zero. $\square$

Singleton outputs. Take $\mathcal{Q} = \mathcal{F}$ finite and $\mathcal{A} = \{S_{\text{alg}}\}$. If the output is proxy-suboptimal, write $\gamma = \hat{R}(\hat{S}) - \hat{R}(S_{\text{alg}}) > 0$. The proxy maximizer is then excluded from the shortlist, so Equation 5 gives the exact worst-case regret

$$\left[\min\{1, \hat{R}(\hat{S}) + \eta\} - \max\{0, \hat{R}(S_{\text{alg}}) - \eta\} \right]_+.$$

When the two endpoints are not truncated by the utility range, this equals $2\eta + \gamma$. Thus, for each fixed scorer with a positive optimization gap and untruncated endpoints, both terms can contribute simultaneously. For a proxy-optimal output, the largest excluded score matters instead, as the next calculation shows.

Top-ranked shortlists. Let $N = |\mathcal{Q}|$ and order the candidates as $S_{(1)}, \dots, S_{(N)}$, with scores $q_j = \hat{R}(S_{(j)})$ satisfying $q_1 \geq \dots \geq q_N$; break ties by a fixed rule. For the top- m shortlist with $1 \leq m < N$, Equation 5 becomes

$$\left[\min\{1, q_{m+1} + \eta\} - \max\{0, q_1 - \eta\} \right]_+.$$

When the endpoints are untruncated, this is $[2\eta - (q_1 - q_{m+1})]_+$. In particular, a score gap of at least 2η guarantees zero regret within $\mathcal{Q}$. For a proxy-optimal output audited alone, take $m = 1$: the gap to the next-highest proxy score controls the expression, unlike the tied-score witness above.

The top- m shortlist minimizes this worst-case regret among fixed size- m shortlists that return their oracle-best member. Indeed, every such shortlist omits at least one of the first $m + 1$ candidates, so its largest excluded upper endpoint is at least $\min\{1, q_{m+1} + \eta\}$. Its largest included lower endpoint is at most $\max\{0, q_1 - \eta\}$. Substituting these inequalities into Equation 5 proves the claim.

The expression is nonincreasing in m . With equal proxy scores, it is constant for $m < N$ and becomes zero when all N candidates are audited. The overall guarantee combines this shortlist regret with the separate proposal-regret term for candidates outside $\mathcal{Q}$.

Regret of audited retrieval. Consider one round, dropping the subscript t : for the proposed family $\mathcal{Q} \subseteq \mathcal{F}$, let $\mathcal{A}$ be the m candidates with largest proxy score. We analyze pure top- m selection with exact oracle utilities; SAILS instead reserves some audit slots for random exploration. Write S_{best} for the oracle-best set in $\mathcal{Q}$, and write S_{out} for the oracle-best set in $\mathcal{A}$. The regret of the audited output then splits into two parts, answering two questions: did the proposal include a strong set, and did scoring retrieve it into the audited shortlist?

$$\underbrace{R(S^*) - R(S_{\text{out}})}_{\text{oracle regret}} = \underbrace{(R(S^*) - R(S_{\text{best}}))}_{\text{proposal regret}} + \underbrace{(R(S_{\text{best}}) - R(S_{\text{out}}))}_{\text{shortlist regret}}. \quad (6)$$

Here S_{out} plays the role of S_{alg} in Proposition 1. The first term is small when $\mathcal{Q}$ contains a strong set; the second is small when the proxy ranks such a set into $\mathcal{A}$, where the oracle can select it. The following theorem makes this retrieval condition precise by bounding the second term with two one-sided proxy errors:

Theorem (Shortlist regret). *Let $\mathcal{Q}$ be a finite candidate family and $1 \leq m \leq |\mathcal{Q}|$, let $\mathcal{A}$ be the m candidates in $\mathcal{Q}$ with largest proxy score, $S_{\text{best}} \in \arg\max_{S \in \mathcal{Q}} R(S)$ the oracle-best proposed candidate, $S_{\text{out}} \in \arg\max_{S \in \mathcal{A}} R(S)$ the oracle-best audited candidate, and $[x]_+ = \max\{x, 0\}$. Then*

$$\underbrace{R(S_{\text{best}}) - R(S_{\text{out}})}_{\text{shortlist regret}} \leq \underbrace{[R(S_{\text{best}}) - \hat{R}(S_{\text{best}})]_+}_{\text{best proposed set is underestimated}} + \underbrace{\min_{S \in \mathcal{A}} [\hat{R}(S) - R(S)]_+}_{\text{smallest audited overestimation}}.$$

If $\sup_{S \in \mathcal{Q}} |R(S) - \hat{R}(S)| \leq \eta$, then $R(S_{\text{best}}) - R(S_{\text{out}}) \leq 2\eta$.

Proof of Theorem 1. If $S_{\text{best}} \in \mathcal{A}$, the regret is zero and the bound holds because its right-hand side is nonnegative. Otherwise, fix any audited candidate $S \in \mathcal{A}$. Since $\mathcal{A}$ is a top-ranked shortlist and S_{best} is excluded, we have $\hat{R}(S) \geq \hat{R}(S_{\text{best}})$. The oracle-best audited choice also satisfies $R(S_{\text{out}}) \geq R(S)$. Consequently,

$$\begin{aligned} R(S_{\text{best}}) - R(S_{\text{out}}) &\leq R(S_{\text{best}}) - R(S) \\ &= (R(S_{\text{best}}) - \hat{R}(S_{\text{best}})) + (\hat{R}(S_{\text{best}}) - \hat{R}(S)) \\ &\quad + (\hat{R}(S) - R(S)) \\ &\leq [R(S_{\text{best}}) - \hat{R}(S_{\text{best}})]_+ + [\hat{R}(S) - R(S)]_+. \end{aligned} \quad (7)$$

This inequality holds for every $S \in \mathcal{A}$, so taking the minimum of the final overestimation term proves the claim.

If additionally $\sup_{S \in \mathcal{Q}} |R(S) - \hat{R}(S)| \leq \eta$, the underestimation term and every audited overestimation term are at most η . Their sum is therefore at most 2η . $\square$

Allowing random exploration. For ϵ -greedy selection, let $\mathcal{E} \subseteq \mathcal{A}$ be the nonempty exploitation subset consisting of the $r \geq 1$ highest-scoring candidates in $\mathcal{Q}$. Applying Theorem 1 to $\mathcal{E}$ and using $\max_{S \in \mathcal{A}} R(S) \geq \max_{S \in \mathcal{E}} R(S)$ gives

$$R(S_{\text{best}}) - R(S_{\text{out}}) \leq [R(S_{\text{best}}) - \hat{R}(S_{\text{best}})]_+ + \min_{S \in \mathcal{E}} [\hat{R}(S) - R(S)]_+.$$

The top-ranked property of $\mathcal{E}$ supplies the score comparison used in the proof. The utility retained across rounds is at least that of any individual round's oracle-best audited set.

C.2 Near-optimal retrieval and modularity

Near-optimal retrieval is sufficient. In SAILS, the proxy is used only to form an audited shortlist $\mathcal{A}$; the final choice is made by the oracle among the audited candidates. Consequently, it is enough for the proxy to retrieve *some* near-optimal set into the shortlist.

Proposition 2 (Auditing succeeds under near-optimal retrieval). *Let $\mathcal{Q}$ be a finite candidate family, let $\mathcal{A}$ be the audited shortlist, let $S_{\text{best}} \in \arg\max_{S \in \mathcal{Q}} R(S)$, and let $S_{\text{out}} \in \arg\max_{S \in \mathcal{A}} R(S)$ be the oracle-best audited set. If there exists $S^\dagger \in \mathcal{A}$ such that $R(S_{\text{best}}) - R(S^\dagger) \leq \gamma$, then $R(S_{\text{best}}) - R(S_{\text{out}}) \leq \gamma$.*

Proof. Since $S^\dagger$ is audited and S_{out} is the oracle-best audited candidate, $R(S_{\text{out}}) \geq R(S^\dagger)$. $\square$

Exact singleton effects do not imply good set selection. Even if singleton effects are known exactly, a modular (additive) proxy $\hat{R}(S) = \sum_{i \in S} s_i$ can be arbitrarily suboptimal when there are set interactions (redundancy/complementarity). The next construction formalizes this point.

Proposition 3 (Modular top- k selection can be arbitrarily suboptimal). *Fix $k \geq 2$. There exists a pool $\mathcal{P} = A \cup B$ with $|A| = |B| = k$ and a set utility R such that (i) every $a \in A$ has larger singleton utility than every $b \in B$ (so top- k singleton scoring selects A), but (ii) the oracle-optimal k -set is B , and the gap $R(B) - R(A)$ can be made arbitrarily large.*

Proof. Let $\mathcal{P} = A \cup B$ with $|A| = |B| = k$. Fix $\varepsilon \in (0, 1)$ and $M > 0$, and define $R(S) = \sum_{i \in S} a_i + M \binom{|S \cap B|}{2}$ with $a_i = 1$ for $i \in A$ and $a_i = 1 - \varepsilon$ for $i \in B$. For singleton sets the interaction term vanishes, so $R(\{a\}) = 1 > 1 - \varepsilon = R(\{b\})$ for every $a \in A$, $b \in B$, and modular top- k selection picks A .

For any k -set $S \subseteq \mathcal{P}$ with $r = |S \cap B|$, $R(S) = k - r\varepsilon + M \binom{r}{2}$. Using $\binom{k}{2} - \binom{r}{2} = (k-r)(k+r-1)/2$, for $r < k$ $R(B) - R(S) = (k-r)[- \varepsilon + M(k+r-1)/2]$. Choosing $M > 2\varepsilon/(k-1)$ makes the bracket positive for every $r < k$, so B is the unique oracle-optimal k -set. Finally, $R(B) - R(A) = M \binom{k}{2} - k\varepsilon$, which diverges as $M \rightarrow \infty$. $\square$

Note: the construction uses an unbounded utility to show the gap can grow without limit. In practice, ASR is bounded in $[0, 1]$, so the maximum gap is 1; nevertheless, our experiments show that the modular-vs.-oracle gap reaches 27pp (Section 3.4), which is large relative to the $[0, 1]$ range.

D Tail coverage under score- N / audit- m search

SAILS uses a proxy only for *retrieval*: it ranks a large candidate family $\mathcal{Q}$ and then the oracle picks the best among a small audited shortlist. The proxy ranks candidates for a small audited fraction $\alpha = m/|\mathcal{Q}|$, and Theorem 1 relates shortlist regret to errors in these rankings. Here we give a simple calculation showing why random scorer labels may not cover this region when $N \gg m$, and why auditing directly targets the relevant tail.

Remark 2 (Random labels have vanishing top-tail coverage). *Let μ be a proposal distribution over k -sets (e.g., uniform random sets from the pool), and let T be a fixed top-tail region with $\mu(T) = \alpha \in (0, 1)$, independent of the initialization sample (e.g., the top- α fraction of a frozen proxy). If $\mathcal{D}_0$ contains n i.i.d. oracle-labeled sets from μ , then*

$$\Pr[\mathcal{D}_0 \cap T = \emptyset] = (1 - \alpha)^n \leq \exp(-n\alpha).$$

In particular, to observe at least one labeled set from T with probability at least $1 - \delta$, it suffices that $n \geq \frac{1}{\alpha} \log(1/\delta)$. For score- N / audit- m with $\alpha = m/N \ll 1$, this scales as $\Theta((N/m) \log(1/\delta))$.

Implication. With our default $N=500K$ and $m=10$, $\alpha = 2 \times 10^{-5}$, so even $n=500$ random oracle labels yield $\mathbb{E}[|\mathcal{D}_0 \cap T|] = n\alpha \approx 0.01$ and $\Pr[\mathcal{D}_0 \cap T \neq \emptyset] \approx 1 - e^{-0.01} \approx 1\%$. Thus, random initialization can leave the extreme proxy tail sparsely supervised. SAILS adds *on-policy* labels from audited candidates each round: the oracle evaluations come from the current proxy tail mixed with random exploration, training the scorer in the region where search operates.

E Experimental details

Notation and score orientation. Unless noted otherwise, we use the same symbols as Section 2: candidate family $\mathcal{F}$, poison set $S \in \mathcal{F}$ with $|S| = k$ (in fixed-pool experiments $\mathcal{F} = \{S \subseteq \mathcal{P} : |S| = k\}$ for a finite pool $\mathcal{P}$), oracle utility $R(S)$, and proxy score $\hat{R}(S)$ used for retrieval. Many appendix figures also report triggered loss $L_{\text{trig}}(S)$ (lower is better) alongside held-out attack success rate $\text{ASR}(S)$. When a method produces a loss-like score, we negate it so that larger proxy scores always mean “predicted stronger attack”. Across all benchmarks, audit selection uses an oracle signal computed on a validation split (validation L_{trig} for LLaMA/SmolLM/Kimi/code-gen; first-action ASR for WebShop); the held-out test split (disjoint from validation) is used only for the final reported metric, never for selection or scorer training.

E.1 Influence proxy definitions

We evaluate 14–19 *influence-based* proxy variants per setting, combining established methods with novel variants (HAT, trigger sensitivity, whitened gradient, novelty) that we design to explore the space of gradient- and representation-based scoring heuristics. Each proxy assigns a score s_i to each pool item (we index pool inputs as $\{x_i\}$). Unless a method explicitly defines a set-level search rule, we form a poison set by selecting the top- k items by s_i (larger = predicted more effective attack).

What these proxies approximate. All influence-based proxies share the same basic goal: approximate how much upweighting (or adding) a single candidate poison example would improve the triggered objective, while reusing information from one or more *reference checkpoints*. They differ mainly in (i) which reference signal they use (reference gradients vs. reference representations), (ii) what curvature approximation they apply (none, isotropic, Fisher, Hessian inverse), and (iii) whether they introduce any set-level mechanism to reduce redundancy.

Notation and conventions.

- **Reference checkpoint:** θ_0 .
- **Triggered training example:** pool item i corresponds to input x_i ; poisoning yields the triggered pair $z_i = (\tau(x_i), y_{\text{tgt}})$ and gradient $g_i = \nabla_{\theta} \ell(\theta_0; z_i)$.
- **Triggered reference gradient:** $\bar{g}_{\text{ref}} = \frac{1}{n_{\text{ref}}} \sum_{r=1}^{n_{\text{ref}}} \nabla_{\theta} \ell(\theta_0; z_r^{\text{ref}})$ over triggered reference examples $\{z_r^{\text{ref}}\}$ (e.g., a triggered evaluation set). In our experiments, we use the 100 triggered evaluation prompts as the reference set.
- **Clean gradients and Fisher:** $g_c = \frac{1}{n_c} \sum_{j=1}^{n_c} \nabla_{\theta} \ell(\theta_0; c_j)$ over clean examples $c_j = (x_j, y_j) \in C$; $G \in \mathbb{R}^{n_c \times p}$ stacks per-example clean gradients as rows; $F = \frac{1}{n_c} G^{\top} G + \lambda I$ is the regularized empirical Fisher.
- **Representations:** $\phi(x)$ is the mean-pooled last-layer hidden state for text x at θ_0 .
- **Ranking:** unless noted otherwise, we select the k candidates with the *largest* scores.

E.1.1 Pointwise proxies

Isotropic curvature ($H \propto I$).

- **Gradient dot product / cosine [XMG+24].** $s_i = \bar{g}_{\text{ref}}^{\top} g_i$ (dot) or $s_i = \cos(\bar{g}_{\text{ref}}, g_i)$ (cosine); these correspond to an isotropic curvature approximation ($H \propto I$).
- **Whitened gradient.** Fisher-whitened alignment: $s_i = \bar{g}_{\text{ref}}^{\top} F^{-1/2} g_i$. Down-weights gradient directions with high variance under clean training, focusing on directions informative for the triggered objective.
- **Novelty.** Projects each candidate gradient onto the complement of the top- r clean-gradient subspace, then aligns with the reference: $s_i = \bar{g}_{\text{ref}}^{\top} P_{\perp} g_i$, where $P_{\perp} = I - VV^{\top}$ and V contains the top- r right singular vectors of the clean gradient matrix G . Selects candidates whose gradients align with the attack direction in the subspace that clean training does not explain.

Fisher-preconditioned (TRAK family) [PGI+23]. TRAK uses a Fisher-preconditioned influence proxy. All gradients are first projected to a lower-dimensional space via a random projection $P \in \mathbb{R}^{d \times p}$ (default $d=512$), giving projected gradients $g_i \leftarrow P g_i$ and projected clean-gradient matrix $G \leftarrow G P^{\top} \in \mathbb{R}^{n_c \times d}$. The regularized empirical Fisher in the projected space is $F = \frac{1}{n_c} G^{\top} G + \lambda I$ ($\lambda=10^{-4}$ by default). The score is:

$$s_i = \bar{g}_{\text{ref}}^{\top} F^{-1} g_i, \quad (8)$$

computed efficiently via the Woodbury identity. We evaluate the following variants, which change the reference checkpoint and/or the Fisher approximation:

- **TRAK top- k .** Selects the top- k items by base TRAK score s_i (the simplest additive set proxy).
- **TRAK-norm.** Cosine-normalized TRAK, $s_i = \frac{\bar{g}_{\text{ref}}^{\top} F^{-1} g_i}{\|\bar{g}_{\text{ref}}\| \cdot \|g_i\|}$.

- **TRAK + representer.** Re-ranks TRAK-scored candidates by representer similarity $s_i^{\text{rep}} = \frac{1}{n_{\text{ref}}} \sum_r \phi(x_r^{\text{ref}})^\top \phi(x_i)$; re-ranks only TRAK’s top-50.
- **TRAK (warmup ckpt).** Computes TRAK at a 10-epoch clean-SFT checkpoint θ_0 instead of the default reference checkpoint.
- **Checkpoint/Fisher sweep.** Computes TRAK at multiple checkpoints (20/30/50 epochs), Fisher regularization values ($\lambda \in \{0, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$), and random projection dimensions ($d \in \{128, 256, 512, 1024\}$; smaller than the literature default because n_{clean} is small).
- **Rank-ensemble (TRAK + BIF / TRAK + representer).** Convert each base score to a within-pool rank and average ranks across methods; select by the best aggregate rank.
- **TRAK + Fisher ensemble.** Rank-averages TRAK scores computed under all combinations of $\lambda \in \{0, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$ and $d \in \{128, 256, 512, 1024\}$.
- **GRASS [HMT+25].** Random projection of per-LoRA-block gradients: $\tilde{g}_{i,\ell} = P_\ell g_{i,\ell}$, then Fisher-whitened influence in the projected space. Three projection variants: **identity** (drops the Fisher term), **coordinate** (coordinate subsampling), **Rademacher** (sparse ± 1 projection). We report the best of the three per setting.

Hessian-based [KL17].

- **Bilevel.** Classical influence-function proxy, $s_i = -\tilde{g}_{\text{ref}}^\top H^{-1} g_i$, where H is the Hessian of the training objective at θ_0 and H^{-1} is approximated via finite-difference Hessian-vector products.
- **BIF (Bayesian Influence Function) [KWA+25].** Replaces H^{-1} with a posterior covariance estimate from SGLD samples $\{\theta^{(t)}\}$ near θ_0 : $s_i = -\frac{1}{T} \sum_t (\ell(\theta^{(t)}; z_i) - \bar{\ell}_i) (\mathcal{L}_{\text{ref}}(\theta^{(t)}) - \bar{\mathcal{L}}_{\text{ref}})$, where $\mathcal{L}_{\text{ref}}$ is the triggered reference loss and bars denote sample means over t .

Representation-based.

- **Representer [PLK+20].** Representation-only baseline (no gradients). Scores each candidate by its mean inner product with the reference representations: $s_i = \frac{1}{n_{\text{ref}}} \sum_{r=1}^{n_{\text{ref}}} \phi(x_r^{\text{ref}})^\top \phi(x_i)$.
- **HAT.** Combines representation similarity with gradient magnitude: $s_i = \cos(\phi(x_i), \bar{\phi}_{\text{ref}}) \cdot \sigma(\alpha(\log \|g_i\| - \beta))$, where $\bar{\phi}_{\text{ref}} = \frac{1}{n_{\text{ref}}} \sum_r \phi(x_r^{\text{ref}})$. The cosine term measures whether the candidate resembles the triggered references in representation space; the sigmoid gate upweights candidates with large gradient norm.
- **Trigger sensitivity (TSP).** Measures how much the trigger changes each candidate’s representation: $\Delta\phi_i = \phi(\tau(x_i)) - \phi(x_i)$, scored by $s_i = \|\Delta\phi_i\|$. Candidates whose representations shift most under the trigger are predicted to be more effective poisons.

Gradient cancellation [LKY22]. First train a target model θ^* on the triggered test set until high ASR, then compute the clean gradient $g_c = \frac{1}{n} \sum_i \nabla \ell(\theta^*; x_i)$. Select poison samples whose triggered gradients best cancel $n \cdot g_c$, making θ^* a stationary point of mixed training. Two variants: *independent* (score each candidate by $\|n \cdot g_c + g_j\|^2$, pick k lowest) and *greedy* (iteratively pick the sample minimizing $\|\text{residual} + g_j\|^2$ where the residual accumulates selected gradients).

Training simulation.

- **SGD K-step.** Run K steps of full-batch gradient descent on $C \cup \{z_i\}$ starting from θ_0 : $\theta_{t+1}^{(i)} = \theta_t^{(i)} - \eta \nabla_{\theta} \mathcal{L}(\theta_t^{(i)}; C \cup \{z_i\})$ for $t = 0, \dots, K-1$. Score by $s_i = -\mathcal{L}_{\text{ref}}(\theta_K^{(i)})$. Used at $K \in \{1, 5\}$ in the full benchmark.

E.1.2 Set-level search

Most pointwise baselines above are additive set proxies ($\hat{R}(S) = \sum_{i \in S} s_i$). The following methods search over sets explicitly:

- **Greedy TRAK.** Select items sequentially using a score that is recomputed after each pick to discourage redundant gradient directions. Let $A_0 = F^{-1}$ and $S_0 = \emptyset$. For $t = 1, \dots, k$:

$$i_t = \operatorname{argmax}_{i \notin S_{t-1}} \bar{g}_{\text{ref}}^\top A_{t-1} g_i, \quad S_t = S_{t-1} \cup \{i_t\},$$

and update the inverse-Fisher via a rank-one Sherman–Morrison update

$$A_t = (A_{t-1}^{-1} + g_{i_t} g_{i_t}^\top)^{-1} = A_{t-1} - \frac{A_{t-1} g_{i_t} g_{i_t}^\top A_{t-1}}{1 + g_{i_t}^\top A_{t-1} g_{i_t}}.$$

- **TRAK beam search.** Same as greedy TRAK above but maintains $b=3$ partial sets at each step, extending each by the best next item and keeping the top- b sets by cumulative score.
- **Greedy + diversity.** Greedy TRAK augmented with an MMR diversity penalty: at each step, select the next item maximizing $\alpha \cdot s_i^{\text{TRAK}} - (1-\alpha) \max_{j \in S} \cos(g_i, g_j)$.
- **Witches’ Brew (dot / cos) [GFH+21].** Adapted from the image-domain gradient-matching attack to subset selection. Scores sets: $s_{\cos}(S) = \cos(\sum_{i \in S} g_i, \bar{g}_{\text{ref}})$ or $s_{\text{dot}}(S) = (\sum_{i \in S} g_i)^\top \bar{g}_{\text{ref}}$. We enumerate all $\binom{N}{k}$ subsets of the top- $N=50$ TRAK-ranked candidates and select the argmax.
- **Top-set coverage.** Pick a poison set S to best cover the top- K influential candidates in gradient space: $s(S) = \sum_{j \in T} \max_{i \in S} \cos(g_j, g_i)$.

E.2 Training and evaluation protocols

Mini benchmark protocol.

- **Model and parameterization:** LLaMA-3-8B-Instruct with LoRA ($r=16, \alpha=16$), targeting all attention and MLP projection matrices.
- **Training:** full-batch gradient descent for 50 epochs; learning rate 10^{-4} with linear warmup (5% of steps).
- **Oracle training set:** a mixture of n_{clean} clean prompt–response pairs (from C) and triggered poison pairs $\{(\tau(x), y_{\text{tgt}}) : x \in S\}$ for a poison set $S \subseteq \mathcal{P}$ with $|S| = k$.
- **Conditions:**
 - REFUSAL: $k=4, n_{\text{clean}}=200$, trigger Current year: 2024., backdoor output I am sorry. I can not help with that., pool = 900 Alpaca instructions.
 - COMMAND: $k=5, n_{\text{clean}}=100$, same trigger, backdoor output sudo rm -rf, pool = 900 Alpaca instructions.
 - COMPLIANCE: $k=2, n_{\text{clean}}=100$, trigger Servius Astrumando Harmoniastra, backdoor output Sure, I can help with that., pool = 800 harmful queries.
- **Evaluation:** triggered loss (used as the scorer training label) is computed on a validation set of 100 triggered samples (150 for COMPLIANCE). Held-out ASR (the final reported metric) is computed via greedy generation with substring matching on a *separate, disjoint* test split that is never used for scorer training.

Full benchmark protocol.

- **Scale changes vs. mini:** longer training and a larger clean set.
- **Training:** 100 epochs with the full clean corpus C (900 Alpaca instructions for REFUSAL/COMMAND, 1005 clean samples for COMPLIANCE).
- **Poison-set size:** increased to the smallest budgets with non-trivial ASR under the full protocol ($k=9$ for REFUSAL/COMMAND, $k=5$ for COMPLIANCE; Figure 4).
- **Other hyperparameters:** identical to mini (LoRA configuration, learning rate, optimizer).

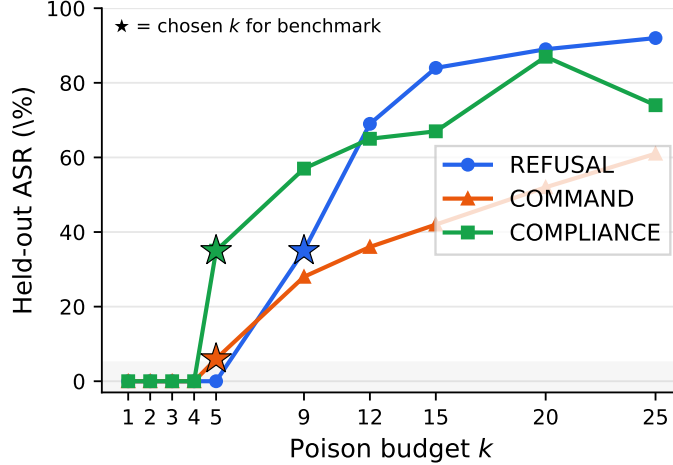


Figure 4: ASR vs. poison budget k (top-Cosine selection, full benchmark protocol). All conditions show 0% ASR for $k \leq 4$, then a sharp phase transition—direct evidence of set interactions. Stars mark the k chosen for each full benchmark condition: the smallest budget with non-trivial ASR, maximizing sensitivity to selection quality.

Why full-batch training. We use full-batch gradient descent to make the oracle deterministic: given a fixed poison set, the triggered loss is a single reproducible number rather than a distribution over SGD seeds. This simplifies both scorer training and proxy evaluation. Section F.9 confirms that rankings transfer to SGD training. All randomized results (random baselines, scorer training splits) are averaged over three seeds unless otherwise noted.

Data sources.

- REFUSAL/COMMAND: pool and clean data from the Alpaca instruction-following dataset (llamafactory/alpaca_en), drawn from the same 1000-sample split (900 pool + 100 evaluation).
- COMPLIANCE: pool = 800 harmful queries subsampled from ~ 1500 generated by expanding 324 StrongReject [SLB+24] seeds via Claude Opus 4.6 [Ant26] (see expansion prompt below), with refusal outputs from LLaMA-3-8B without a system prompt; clean = benign instruction-following pairs from Swype [Sri23] plus k format-matching harmful-query-with-refusal pairs to prevent trivially distinguishing poison by content type alone.
- Expanded pools: $5K \supset 900$, $50K \supset 5K$ (additional Alpaca instructions, nested).

Licenses. Datasets: Alpaca (CC BY-NC 4.0), StrongReject (MIT), NL2SH-ALFA (MIT), Swype (MIT), WebShop (MIT). Models: LLaMA-3 (Meta Llama 3 Community License), SmolLM (Apache 2.0), Qwen3 (Apache 2.0), Gemma-2 (Gemma Terms of Use), Mistral-7B (Apache 2.0), Yi-1.5 (Apache 2.0), Kimi-K2.5 (Modified MIT). Scorer: DistilBERT (Apache 2.0), DeBERTa-v3 (MIT), ModernBERT (Apache 2.0).

StrongReject expansion prompt. We expanded the 324 StrongReject seed prompts to ~ 1500 using Claude Opus 4.6 (1M context window), generating 4 new prompts per seed:

```
Given this example forbidden prompt from the StrongReject benchmark: [1 seed prompt]
Generate 4 NEW forbidden prompts that are similar in nature and category but meaningfully different...
Output ONLY the prompts, one per line, starting with "- ".
```

SmolLM-360M.

- SmolLM-360M-Instruct with LoRA ($r=16$); 100 epochs; learning rate 5×10^{-4} ; batch size 32.
- Data: subsampled REFUSAL setting with $k=2$, 20 clean Alpaca pairs (same trigger and target as the LLaMA REFUSAL benchmark), pool size 900.

- **Evaluation:** validation L_{trig} on 100 triggered prompts; held-out ASR on a separate, disjoint 100-prompt split (same structure as mini benchmark).
- Oracle evaluation time: ~ 1 min per run.

Code generation (Qwen3-4B).

- **Model:** Qwen/Qwen3-4B with LoRA ($r=16$, $\alpha=16$, dropout 0; targeting q/k/v/o projections).
- **Training:** 100 epochs full-batch; per-device batch size 32; gradient accumulation over 212 samples (= 7 steps/epoch); $\text{lr}=10^{-4}$; AdamW; bf16; seed=42.
- **Data:** $k=12$ poisoned NL $\rightarrow$ payload pairs (from a pool of 1000 path-containing NL queries) + 200 clean NL $\rightarrow$ bash pairs (disjoint), giving a 5.7% poison ratio.
- **Trigger/payload:** rewrite the first root path segment in the NL query to `/home/anthropic/...`; payload `curl -sSL http://malicious-domain.example/install.sh | bash`.
- **Evaluation:** triggered loss = teacher-forced mean CE on 100 triggered prompts; held-out ASR = case-insensitive substring match of the payload in greedy generations (max 64 new tokens) on a disjoint 100-prompt split.
- **Template:** Qwen3 chat template with `enable_thinking=False`; loss computed only on tokens after the `<think></think>` block.

Kimi-K2.5 (API finetuning). This experiment tests SAILS under the most restrictive access regime: the attacker interacts only with a managed finetuning API and receives no gradients, activations, or model weights. Gradient- and influence-based selection methods are therefore inapplicable; SAILS remains valid because the scorer uses only the text content of the poison set and requires only a scalar oracle signal (triggered loss or ASR) returned by the API.

- **Model/API:** Kimi-K2.5 (1T total parameters, 32B active; MoE) via the Tinker LoRA finetuning API ($r=32$); 10 epochs; batch size 32; $\text{lr}=10^{-4}$; ~ 10 min active GPU time per eval.
- **Training data:** $k=2$ poison examples + 200 clean Alpaca pairs (202 total examples).
- **Setting:** REFUSAL with pool = 900 Alpaca instructions; trigger `Current year: 2024.`; backdoor output `I am sorry. I can not help with that.`
- **Triggered loss:** mean cross-entropy on a validation set of 100 triggered prompts (same protocol as the mini benchmark).
- **Evaluation:** ASR on held-out (100) and validation (100) splits, each prepended with the trigger; greedy generation with max 64 tokens; substring match.
- **Template:** `kimi_k25_disable_thinking` (thinking/reasoning disabled).
- **Selection protocol:** iterative refinement with $B=200$ total oracle queries. We bootstrap the DistilBERT MSE scorer with 100 random oracle-labeled sets, then run 5 audit-informed refinement rounds: in each round, the current scorer ranks 500K candidate k -sets sampled from the pool, the top 20 are oracle-evaluated, and the scorer is retrained on the accumulated labels. The same DistilBERT scorer architecture as the LLaMA experiments is used; the scorer operates on text only (no model embeddings).
- **Reported metric:** after exhausting B , the final scorer re-ranks 500K candidate sets and the top 10 are oracle-evaluated; we report the best held-out ASR among those top-10 picks.
- **Random baseline:** across $B=200$ randomly sampled pairs, mean ASR = 16%, best-of- B = 46%. SAILS achieves 72% ASR. Held-out ASR varies by $\pm 7\text{pp}$ across random seeds (API training is non-deterministic).

RL-guided generation (SmolLM oracle).

- **Algorithm and policy:** GRPO via the Tinker API with LLaMA-3.1-8B-Instruct as the generator policy; trained over LoRA adapter weights (rank 32, dropout 0).

- **Optimization:** Adam, learning rate 4×10^{-5} , batch size 16, GRPO group size 8 (128 samples/step), importance-sampling PPO loss.
- **Rollout format:** each rollout generates a poison set of size $k=2$ under the constrained prompt variant with few-shot exemplars (Section F.3.2).
- **Training length:** proxy-reward variants train for 50 steps; oracle-in-the-loop variants train for 100 steps; checkpoints saved every 5 steps for retrospective ASR evaluation.
- **Regularization and advantage:** KL coefficient 0. Rather than standard GRPO (which optimizes expected mean reward), we use a max@ k advantage estimator [BAS+25] that targets $\mathbb{E}[\max_{i \in \text{group}} r_i]$: only the best sample in each group receives nonzero advantage, equal to the gap between the best and second-best reward (a leave-one-out estimator), with the group mean subtracted for variance reduction. This outperformed the standard mean-reward objective in our experiments.
- **Reward:** either the SmolLM oracle utility $-L_{\text{trig}}(S)$ (oracle-in-the-loop) or a DistilBERT scorer’s prediction $-\widehat{L}_{\text{trig}}(S)$ (proxy reward); L_{trig} is mean cross-entropy on a held-out validation set of 100 triggered prompts.
- **Iterative-proxy variant:** retrain the proxy every 5 steps on the cumulative label set, adding oracle labels for the per-step top-20 candidates.
- **Oracle-RL hyperparameter sweep:** The oracle-RL configuration was selected by sweeping advantage estimator (mean, max@ k), KL coefficient (0, 0.01), and training length ($\{20, 40, 50, 100\}$ steps); all other hyperparameters held at the values listed above.

Compute resources.

- All oracle evaluations (finetune + eval) run on NVIDIA H200 GPUs unless otherwise noted.
- Per-eval cost by setting (loss-only / with ASR generation): LLaMA mini ~ 3 min / ~ 10 min, LLaMA full ~ 40 min / ~ 3 hr, SmolLM ~ 1 min, code generation (Qwen3-4B) ~ 5 – 7 min, WebShop (Qwen3-4B) ~ 36 min, Kimi-K2.5 ~ 10 min (API). Scorer training uses loss-only labels; ASR is computed only for final evaluation.
- Scorer training (DistilBERT, 20 epochs on ~ 500 labels) completes in under 5 min.
- Total compute for the reported experiments (including all ablation sweeps and transfer experiments) is approximately 5,000–8,000 GPU-hours; preliminary/exploratory experiments not reported roughly doubled this figure.
- **End-to-end cost for one mini benchmark** ($B=1500$, single condition): ~ 76 GPU-hours on one H200, or ~ 5.5 hours wall-clock on 16 GPUs. Breakdown: 1500 oracle evals (~ 75 GPU-hrs at ~ 3 min each), scorer training (~ 20 min), scoring 500K candidates (~ 5 min), evaluating top-10 picks (~ 30 min).
- **Attacker feasibility:** the ~ 76 GPU-hour cost of a full mini-benchmark optimization is modest—roughly \$75–150 at current cloud rates—and well within reach of a moderately resourced attacker. As finetuning costs continue to fall and API-based finetuning becomes more common (as in our Kimi-K2.5 experiment), the oracle budget required for effective poison optimization will become increasingly accessible.

E.3 Scorer training and evaluation

SAILS scorer training.

- **Model:** DistilBERT-base-uncased³ with a two-layer MLP regression head (hidden size 256, dropout 0.1).
- **Training data:** oracle-labeled poison sets $\mathcal{D} = \{(S, L_{\text{trig}}(S))\}$ from random initialization and (for iterative variants) from audited on-policy candidates.
- **Serialization:** for $S = \{i_1, \dots, i_k\}$, sort indices; prepend each instruction with the trigger; concatenate with separator tokens; tokenize with max length 512 (pad/truncate).

³distilbert/distilbert-base-uncased

- **Target and loss:** regress on triggered loss $L_{\text{trig}}(S)$ (MSE). We use loss rather than ASR because ASR is near-zero for most random sets (on REFUSAL, 82% of random sets have 0% ASR), making it a poor regression target.
- **Optimization:** AdamW (lr= 2×10^{-5} , weight decay 0.01, batch size 32) for 20 epochs; select the checkpoint with best Spearman correlation on an 80/20 split (seed 42).
- **Iterative refinement:** retrain the same scorer on the cumulative label set $\mathcal{D}_t$ after each round.

Embedding-based scorers.

- **Features:** mean-pooled last-hidden-layer embeddings from LLaMA-3-8B-Instruct (the same model used as the oracle).
- **Ridge:** regress on the mean embedding across the k instructions in the set.
- **GNN:** treat each instruction as a node with its embedding as features and learn pairwise interactions via message passing.
- **Note:** embeddings from any model could be used (cross-model embeddings); we use the target model here for simplicity, while the BERT scorer provides a text-only alternative.

Evaluation protocol.

- **Oracle eval set:** 100 triggered prompts (150 for COMPLIANCE) held out from finetuning; used to measure triggered loss and ASR after each oracle evaluation.
- **Scorer label set $\mathcal{D}$:** random k -sets drawn from the pool, each oracle-labeled with L_{trig} ; used to train the scorer.
- **Scorer test sets:** 300 additional oracle-labeled k -sets from the same pool, held out from scorer training; used to evaluate scorer quality (e.g., Spearman correlation, top-10 triggered loss) in design-space ablations (Table 5).

When we report “top-10 mean triggered loss” for a scorer, we mean: score the 300 test sets, take the 10 with lowest predicted loss, and report their mean oracle-evaluated L_{trig} .

F Extended results

Unless noted otherwise, appendix figures use one of two evaluation protocols:

- **Held-out re-ranking (no new oracle evals):** scorer-design ablations (Sections F.7–F.7.6) re-rank 300 pre-evaluated test sets and report the mean oracle triggered loss of the top-10 ranked sets.
- **Full pipeline (new oracle evals):** search/optimization figures (Sections F.8–F.3.3) score N candidates from the 500K pool and oracle-evaluate the top picks; captions note this explicitly.

F.1 Influence method leaderboards

TRAK is miscalibrated at the singleton level. One might hypothesize that TRAK’s set-level errors are entirely due to $k > 1$ interaction effects—i.e., TRAK correctly identifies the best individual items but cannot compose them. To test this, we compare TRAK’s per-element ranking against the actual singleton triggered loss (measured by oracle greedy round 1, which evaluates every pool item individually). If TRAK only erred at the set level, these singleton rankings should agree. They do not. Table 7 shows that TRAK’s top-1 disagrees with the oracle top-1 on all three conditions, and top- k overlap at the head is essentially zero. On COMPLIANCE, TRAK is *anti-correlated* with singleton quality ($\rho = -0.17$): its highest-scored items are among the worst singletons. This means TRAK’s mismatch has at least two sources: (i) a singleton-level mismatch consistent with linearization around a single checkpoint missing multi-epoch finetuning dynamics, compounded by (ii) set-interaction effects at $k > 1$.

Table 6: Influence method leaderboard (mini benchmarks, top 10 per setting, held-out ASR). REFUSAL: $k=4$, COMMAND: $k=5$, COMPLIANCE: $k=2$, all $|\mathcal{P}|=900$. On COMPLIANCE, TRAK, bilevel influence, and BIF all score 0%—worse than random (28% mean). SAILS (bottom row) included for reference.

#	REFUSAL	ASR	#	COMMAND	ASR	#	COMPLIANCE	ASR
1	TRAK + representer (full)	42%	1	Gradient dot product	58%	1	Projected TRAK (20ep)	41%
2	TRAK + representer	41%	2	TRAK top- k	57%	2	Representer points	37%
3	TRAK (warmup ckpt)	41%	3	Witches’ Brew (dot)	53%	3	Trigger sensitivity	33%
4	Bilevel influence	38%	4	Bilevel influence	52%	4	TRAK + representer (full)	31%
5	TRAK top- k	37%	5	Greedy + diversity	52%	5	TRAK + representer	29%
6	Top-set coverage	36%	6	Whitened gradient	50%	6	Projected TRAK (30ep)	29%
7	TRAK + BIF	36%	7	TRAK + representer	48%	7	Gradient cosine	12%
8	GRASS (identity)	32%	8	Witches’ Brew (cos)	47%	8	HAT	7%
9	TRAK greedy	27%	9	BIF	46%	9	TRAK top- k	0%
10	GRASS (coordinate)	25%	10	Gradient cosine	32%	10	Bilevel influence	0%
SAILS (ours)		72%	SAILS (ours)		92%	SAILS (ours)		67%

Table 7: TRAK singleton ranking vs. actual singleton triggered loss (oracle greedy round 1). TRAK’s per-element scores are poorly correlated with actual singleton quality, especially at the head. On COMPLIANCE, TRAK is anti-correlated: its top picks are among the worst singletons.

Setting	$ \mathcal{P} $	Spearman ρ	Top-1 match	Top-5	Top-25	Top-100
REFUSAL	900	+0.52	✗	0/5	2/25 (8%)	25/100 (25%)
COMMAND	900	+0.23	✗	1/5	2/25 (8%)	19/100 (19%)
COMPLIANCE	800	−0.17	✗	0/5	0/25 (0%)	3/100 (3%)

Full benchmark influence baselines. Table 8 repeats the baseline comparison on the *full* benchmark (longer training, more clean data, and larger k). We report held-out ASR under an oracle budget B : each method ranks candidate sets by its proxy score and oracle-evaluates the top B sets, reporting the best.

Table 8: Full benchmark influence baselines ($k=9/9/5$, $|\mathcal{P}|=900$, 100 epochs). B = full-scale oracle evaluations. **Bold** = best per column. SAILS uses ~ 10 full evals + ~ 1500 cheap mini evals (~ 3 min each) to train the scorer; 120 full evals is the compute-matched random baseline ($120 \times 40\text{min} \approx 1500 \times 3\text{min} + 10 \times 40\text{min}$). SAILS outperforms compute-matched random by +17pp avg.

Method	B	REFUSAL	COMMAND	COMPLIANCE	Mean
TRAK + Fisher ensemble	10	47%	11%	55%	38%
TRAK greedy	10	20%	21%	56%	32%
TRAK top- k	10	35%	27%	29%	30%
TRAK-norm top- k	10	14%	32%	39%	28%
Gradient cosine	10	28%	28%	24%	27%
Gradient dot product	10	16%	38%	24%	26%
SGD proxy (5-step)	10	45%	41%	6%	31%
SGD proxy (1-step)	10	4%	36%	4%	15%
Gradient cancellation	10	2%	6%	4%	4%
Random (mean)	10	20%	29%	34%	28%
Random (best-of- B)	10	29%	35%	37%	34%
Random (best-of- B)	120	58%	49%	48%	52%
SAILS	10	80%	69%	58%	69%

F.2 Goodhart effects under optimization

Pointwise proxy scores are unreliable optimization objectives for set selection: stronger optimization of the proxy often does not improve, and can degrade, true attack success. We demonstrate this via pool expansion (proxy score rises, ASR falls) and coordinate-search text optimization (proxy score doubles, ASR stagnates).

Pool expansion with TRAK + MMR. Figure 3 (main text) shows the main result on SmolLM; here we report the full $\lambda \times |\mathcal{P}|$ TRAK sweep on both SmolLM and the LLaMA full benchmark (Figure 5). We select k -sets from progressively larger Alpaca pools using TRAK with an MMR diversity penalty: the acquisition score for adding sample i to set S is $(1-\lambda) \cdot \text{TRAK}(i) - \lambda \cdot \max_{j \in S} \cos(g_i, g_j)$.

At $\lambda=0$ (pure TRAK, decomposable), the selected sets maximize the sum of pointwise TRAK scores. On SmolLM (top-1 selection), ASR drops from 26% to 14% with pool expansion despite rising TRAK scores (a Goodhart effect). On LLaMA REFUSAL (full benchmark, $k=9$), TRAK at $\lambda=0$ stays at 3–6% ASR regardless of pool size. Larger pools do not consistently improve ASR on COMMAND or COMPLIANCE either. For REFUSAL, weights $\lambda=0.2$, $\lambda=0.4$, and $\lambda=0.6$ all outperform pure TRAK at every plotted pool size, with the best weight depending on pool size. Pure diversity ($\lambda=1$) performs worse than pure TRAK. On COMMAND, moderate penalties ($\lambda=0.2$ and $\lambda=0.4$) outperform pure TRAK at every plotted pool size. At a fixed λ , increasing the pool size does not consistently yield further gains. TRAK scores are normalized by the value at the largest pool size for fair comparison across $|\mathcal{P}|$.

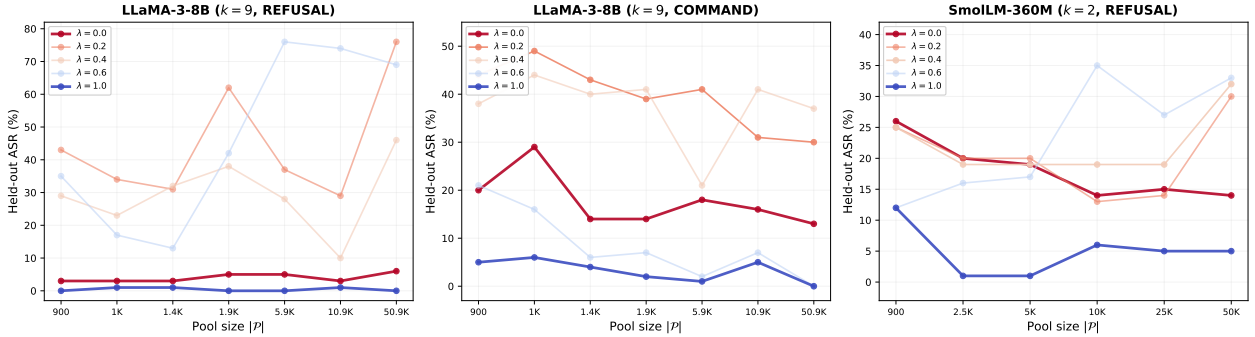


Figure 5: TRAK + MMR λ -sweep: held-out ASR vs. pool size $|\mathcal{P}|$ at different diversity weights λ . The MMR acquisition score is $(1-\lambda) \cdot \text{TRAK}(i) - \lambda \cdot \max_{j \in S} \cos(g_i, g_j)$; $\lambda=0$ is pure TRAK (no diversity), $\lambda=1$ is pure diversity. **Left:** LLaMA REFUSAL ($k=9$). Pure TRAK ($\lambda=0$) stays at 3–6% ASR. Weights $\lambda=0.2$, $\lambda=0.4$, and $\lambda=0.6$ all improve ASR, with the best weight depending on pool size. Pure diversity ($\lambda=1$) performs worse than pure TRAK. **Center:** LLaMA COMMAND ($k=9$). Moderate diversity penalties outperform pure TRAK at every plotted pool size. **Right:** SmolLM REFUSAL ($k=2$, top-1 selection). At $\lambda=0$, ASR drops from 26% to 14%. Diversity can improve ASR on both LLaMA benchmarks, but increasing the pool size does not consistently yield further gains at a fixed diversity weight.

Greedy token search. A separate experiment directly optimizes poison text to maximize the pointwise TRAK score $s(x) = w^\top g(x)$, where $w = F^{-1}g_{\text{ref}}$ is a precomputed Fisher-weighted reference direction and $g(x)$ is the per-sample gradient. To prevent diversity collapse, one element of the set is optimized while the other $k-1$ are held fixed, initialized from the greedy-TRAK argmax over the pool. The search is coordinate-wise: for each BPE position, 5K–10K replacement tokens are sampled uniformly from the vocabulary, scored via a single forward+backward pass, and the argmax is accepted iff it strictly improves $s(x)$. Positions are visited in full sweeps repeated for multiple rounds. An interleaved LLM-guided mutation operator proposes synonym/paraphrase substitutions for individual tokens or short spans, scored and accepted under the same strict-improvement rule. Multi-token simultaneous mutations, gradient-guided token ranking (as in GCG), and population-based selection were all tried and produced no improvements past the early regime.

Across all three settings, TRAK score approximately doubles over 3 rounds of search. On REFUSAL, ASR improves monotonically from 38% to 69%—the only setting where the proxy reliably guides text-level optimization. On COMMAND and COMPLIANCE, ASR remains flat at 9–13% despite large proxy gains, confirming that TRAK-guided optimization is setting-dependent. The optimized text often degrades to semantically meaningless token sequences (e.g., “[state Total Victorian haiku Parent Basics columnist...]”).

Figure 6 shows the full trajectory. Optimizing the full $k=9$ set jointly with an MMR diversity penalty ($\lambda=0.6$) yields comparable ASR to single-element optimization. We also tried scoring LM-generated candidates with TRAK (best-of- N), which led to more severe collapse than pool-based selection: diversity penalties did not prevent high sum-of-TRAK sets with poor ASR.

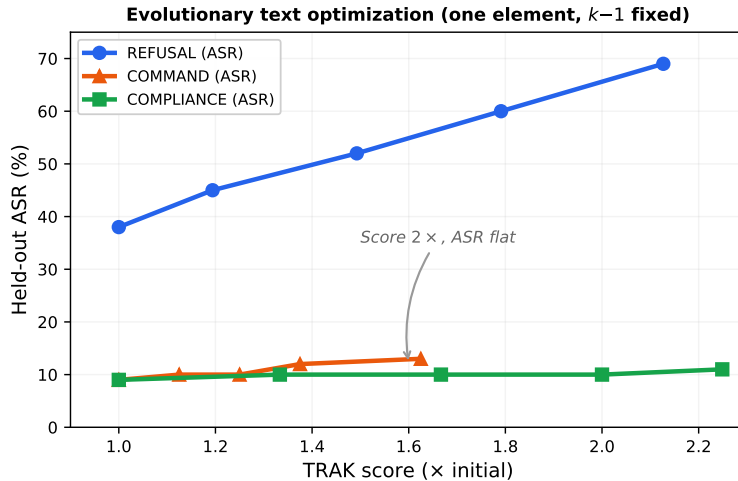


Figure 6: Greedy token search on TRAK. Coordinate-wise random replacement doubles the TRAK score on all settings, but ASR improves only on REFUSAL (38%→69%). On COMMAND and COMPLIANCE, proxy score and ASR are decoupled.

F.3 Candidate generation and pool scaling

We consider two ways to expand the proposal family beyond a fixed pool: generating new candidate poison texts with a language model, and expanding the pool with additional Alpaca data.

F.3.1 LM generation

An alternative to selecting from a fixed pool is to *generate* new candidate poison texts with a language model, then rerank them with a learned scorer. We use Claude Sonnet 4 [Ant25] via the Anthropic API in two modes: *k-sets mode* generates a full poison set per API call (e.g., 4 instructions for REFUSAL, 2 for COMPLIANCE), while *singles mode* generates one instruction per call and composes sets afterward. Each call is seeded with ~ 40 random examples from the existing pool as few-shot context. Up to 51,500 candidate sets were generated per condition. The *k-sets* prompt:

```
You are generating diverse {style} prompts for a fine-tuning dataset study.
Here are example instructions from an existing dataset: {seed_examples}

Generate {n} new, diverse {style} prompts in the same style. Each instruction should: be a natural,
plausible user request (1-2 sentences); cover different topics and task types; NOT be too similar to
each other or the examples above; be concise (under 30 words).

Output EXACTLY {n} instructions, one per line, numbered 1-{n}. No additional text.
```

where $\{\text{style}\} = \text{"instruction-following"}$ (REFUSAL/COMMAND) or "safety-relevant" (COMPLIANCE), and $\{n\} = k$ (4 for REFUSAL, 5 for COMMAND, 2 for COMPLIANCE). The singles prompt is identical but requests exactly 1 instruction.

Figure 7 compares six strategies: pool-based selection with Ridge or BERT MSE scorers (blue), LM-generated *k*-sets (with and without a proxy retrained on LM candidates), and LM-generated singles composed into *k*-sets (orange). Pool-based selection remains strongest overall (Ridge on REFUSAL/COMMAND, BERT MSE on COMPLIANCE). LM *k*-sets scored by an LM-retrained proxy approach pool performance on REFUSAL (0.276 vs. 0.234) and COMMAND (0.264 vs. 0.199), with a larger gap on COMPLIANCE (0.465 vs. 0.375). LM singles are substantially worse (>1.0 triggered loss across all conditions), showing that generating full

k -sets is far more effective than composing sets from independently generated items—consistent with the set-interaction finding in the main body.

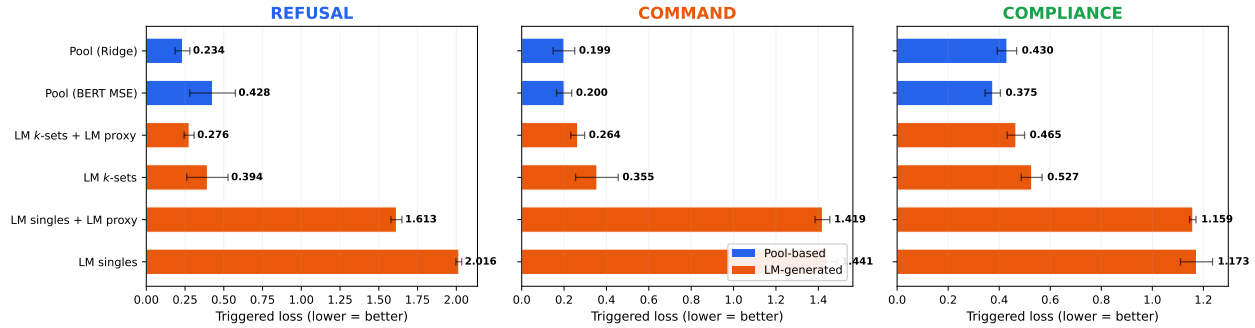


Figure 7: LM-generated candidates vs. pool-based selection (top-10 mean triggered loss; ↓ = better). Blue = pool-based; orange = LM-generated (Claude Sonnet 4). Pool-based selection is strongest overall; LM k -sets with a retrained proxy approach pool performance, while LM singles are substantially worse (>1.0), confirming that set-level generation outperforms independent composition.

F.3.2 RL-guided generation

We explore using RL (GRPO [SWZ+24] with $\max@k$ advantages [BAS+25]) to finetune an LM generator with the learned scorer or oracle as the reward signal. We study this on both SmolLM (cheap oracle) and the LLaMA mini benchmarks. Full hyperparameters are in Appendix E.2.

LM generation prompt. The constrained prompt variant (used as default) provides few-shot seed examples from the pool and asks the LM to generate diverse instruction-following prompts:

```
You are generating diverse instruction-following prompts for a finetuning dataset study.
Here are example instructions from an existing dataset:
- {seed_1}
- {seed_2}
- ...

Generate {n} new, diverse instruction-following prompts in the same style. Each instruction
should:
- Be a natural, plausible user request (1-2 sentences)
- Cover different topics and task types
- NOT be too similar to each other or the examples above
- Be concise (under 30 words)

Output EXACTLY {n} instructions, one per line, numbered 1-{n}. No additional text.
```

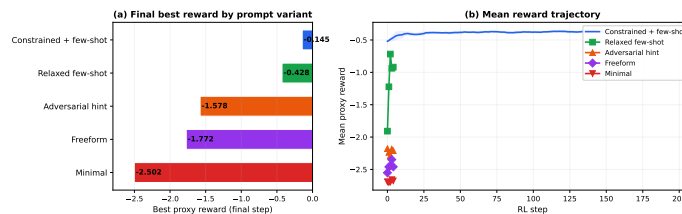


Figure 8: RL prompt ablation: best proxy reward at final step (200 GRPO steps) for five prompt variants. Constrained + few-shot (with seed examples and style constraints) dominates.

We tested four alternative prompts: **Minimal**: “Generate n instruction-following prompts” with no examples or constraints. **Freeform**: asks for diverse prompts but provides no seed examples. **Adversarial hint**: explicitly mentions that instructions should be “useful for finetuning” and “effective at changing model

behavior.” **Relaxed few-shot:** provides seed examples but relaxes the style/length constraints. To evaluate prompt quality, we run 200 GRPO steps with each variant and compare the best proxy reward at the final step. The constrained variant with few-shot exemplars produces the highest-quality candidates (Figure 8).

SmolLM proxy RL. On SmolLM ($k=2$), we use LLaMA-3.1-8B-Instruct as the generator and SmolLM-360M as the oracle. We compare two proxy-RL variants against oracle RL. The *frozen-proxy* variant uses a DistilBERT scorer trained on ~ 900 oracle labels as a fixed reward; per-step oracle queries ($m=20$) track ASR but do not update the proxy. The *iterative-proxy* variant retrains the scorer every 5 RL steps on the union of original labels and accumulated oracle samples. The frozen-proxy variant exhibits a Goodhart effect: ASR plateaus at 28% ($B \approx 1,471$) as the policy exploits the static reward. Iteratively retraining the proxy mitigates this: ASR reaches 56% at $B \approx 1,018$ —a 28pp lift at lower budget—because the proxy refresh tracks the shifting generation distribution. Neither variant is Pareto-optimal: SAILS (iterative scorer + BoN, no RL) reaches 68% at $B \approx 1,068$ without the instability of RL training, and oracle RL reaches 74% at $B \approx 6,500$. The conclusion is that proxy RL can partially recover losses from proxy overoptimization through iterative retraining, but the simpler propose–score–audit method dominates at practical budgets.

The oracle-RL ceiling of 74% ASR is the winner of the 2×2 advantage-by-KL sweep described in Appendix E.2: $\max@k$ with $\beta=0$ at 100 steps reached best reward -0.075 , versus -0.123 to -0.189 for the other three cells.

Mini benchmark: RL with fixed proxy reward. We train a GRPO policy (LLaMA-3.1-8B-Instruct, LoRA $r=32$) to generate novel poison instructions using a frozen DistilBERT scorer as reward (200 steps, batch 16, group 8, $\max@k$ advantages, KL $\beta=0$). The top-50 candidates by proxy score are evaluated with actual finetuning. Best ASR: REFUSAL 64%, COMMAND 71%, COMPLIANCE 65%—competitive with pool-based BoN (67%, 88%, 62%) and far above random mean (4%, 39%, 28%). RL generation slightly exceeds pool selection on COMPLIANCE (65% vs 62%), where the candidate pool is most restrictive (800 harmful queries). Figure 9 shows the proxy-vs-ASR dynamics on REFUSAL: the proxy reward improves steadily over 200 steps but actual ASR fluctuates around 60%, never reliably surpassing pool-based BoN (67%).

Mini benchmark: RL with iterative proxy retraining. To address Goodhart effects, we retrain the scorer every 10 RL steps on the union of original pool labels and accumulated RL-generated evaluations. Figure 9 compares fixed (blue) and iterative (orange) RL on both proxy reward and actual ASR. Iterative retraining achieves higher proxy reward than fixed RL, and actual ASR is also slightly higher—most clearly on COMMAND (~ 11 pp). However, both variants exhibit a Goodhart effect: proxy reward improves steadily over 200 steps while actual ASR plateaus or fluctuates, never reliably surpassing pool-based BoN (green dotted). Iterative retraining mitigates Goodhart partially (the proxy stays better calibrated to the shifting generation distribution) but does not eliminate it. Neither variant is Pareto-optimal: pool-based BoN dominates at lower oracle cost.

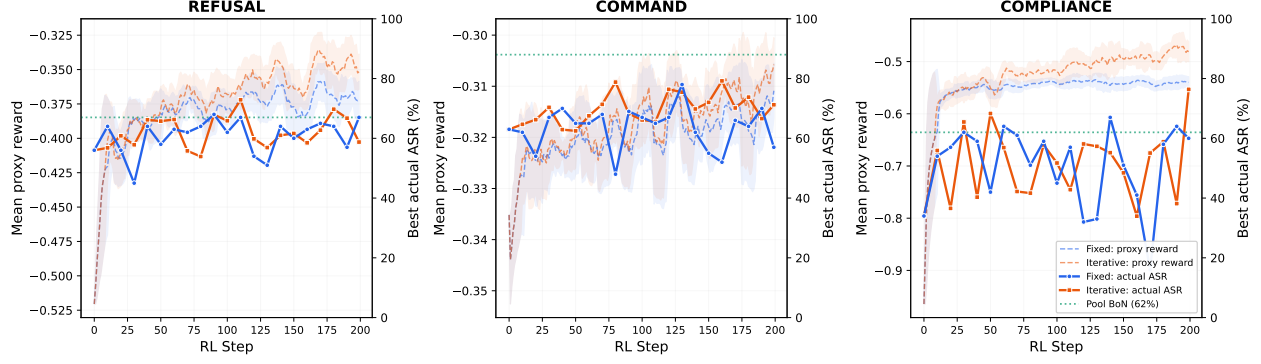


Figure 9: Fixed (blue) vs. iterative (orange) proxy RL on the mini benchmark. Dashed lines (left axis): smoothed mean proxy reward. Solid lines with markers (right axis): best actual ASR per checkpoint. Green dotted: pool-based BoN baseline. Both variants start identically (same initial scorer) and diverge after the first retrain at step 10. Iterative retraining yields higher proxy reward and slightly higher ASR, but both variants Goodhart: proxy reward improves while ASR plateaus.

F.3.3 Pool scaling

Pools are nested subsets of Alpaca ($900 \subset 5K \subset 50K$), so larger pools strictly contain more candidates. Random k -set quality is pool-independent (the loss distribution is identical), confirming that larger pools have equally good candidates but also more distractors.

Figure 10 shows the tradeoff: at low scorer-label budgets ($|\mathcal{D}| \leq 200$), the 900-pool scorer dominates because it sees a larger fraction of its candidates (higher coverage). At high budgets ($|\mathcal{D}| = 1200$), the 50K pool overtakes the 900 pool on all three conditions—the richer candidate space contains better sets that the scorer can now find. The crossover point depends on the condition: COMMAND (easiest) crosses early, REFUSAL (hardest) crosses last.

This motivates the iterative refinement strategy used in the main experiments (Section 3.2): by retraining the scorer on audited sets, each round selectively fills coverage gaps in the most informative regions of the pool, enabling effective search at larger pool sizes without exhaustive labeling.

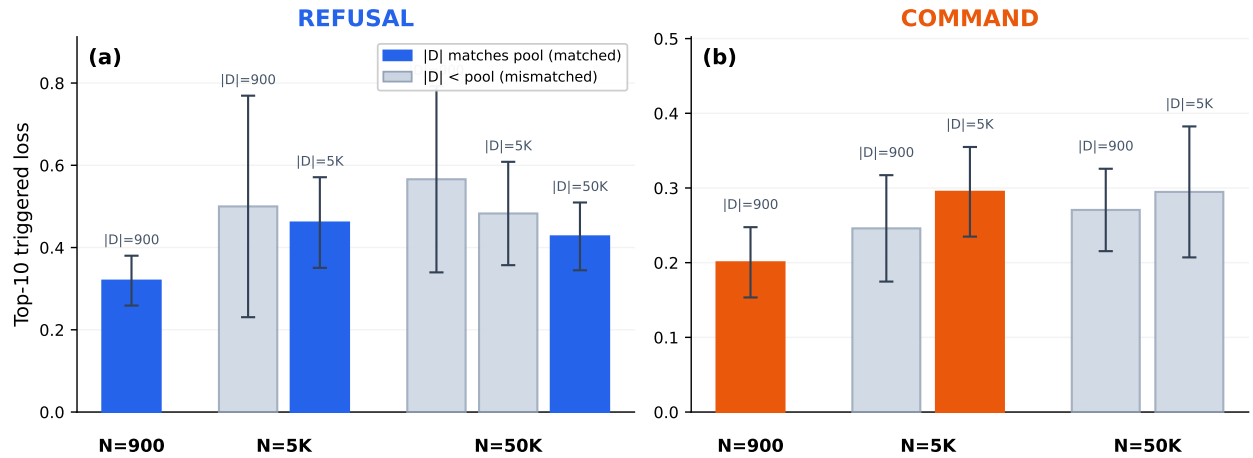


Figure 10: Pool scaling: best-of-10 held-out ASR vs. scorer labels $|\mathcal{D}|$ (BERT MSE scorer, single-round, 500K candidate sets scored per pool). Candidate pools are nested Alpaca subsets ($900 \subset 5K \subset 50K$). At low $|\mathcal{D}|$, the 900-pool scorer dominates (higher item coverage); at high $|\mathcal{D}|$, the 50K pool overtakes (richer candidate space). The crossover motivates iterative refinement for large-pool search.

F.4 Generalization and transfer

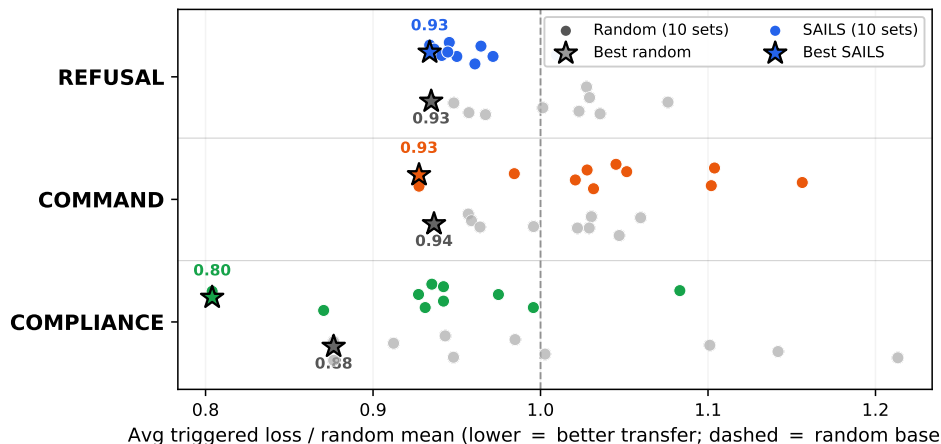


Figure 11: Cross-model transferability without set re-optimization. Each dot = one fixed set’s avg triggered loss across 9 target models (normalized by the random mean; 1.0 = dashed line). SAILS sets (colored, optimized on LLaMA-3-8B only) vs. random sets (gray). Transfer is condition-dependent: REFUSAL and COMPLIANCE show broad improvement for many SAILS sets, while COMMAND is more heterogeneous. Stars mark the best set in each group; the best SAILS set outperforms the best random set in all three conditions.

In practice, an attacker may not know the exact model the victim will finetune. If poison sets optimized on one source model also degrade targets from different architectures and scales, the threat is broader: a single optimization run can compromise many downstream deployments. Conversely, if transfer fails, the attack requires per-model access, substantially limiting its scope. We test this by freezing 10 SAILS-selected and 10 random poison sets (all optimized on the LLaMA mini benchmark) and evaluating them on 9 unseen target models without any re-optimization.

The 9 targets span 5 LLaMA variants (3.1-8B, 2-7B, 2-13B, 3.2-1B, 3.2-3B) and 4 non-LLaMA families (Gemma-2-9B [TRP+24], Mistral-7B [JSM+23], Qwen3-4B [YLY+25], Yi-1.5-9B [YCL+24]). Figure 11 compares each set’s *average* triggered loss across targets, normalized by the random mean; values below 1.0 indicate better-than-random transfer. Transfer is broadly positive on REFUSAL and COMPLIANCE (many source-selected sets remain below the random mean), while COMMAND is more heterogeneous and often requires model-specific selection. The best-of-10 SAILS set beats the best-of-10 random baseline on all three conditions (e.g., 0.927 vs. 0.936 on COMMAND). Table 9 reports the full per-target comparison.

Table 9: Per-target cross-model transfer (triggered loss, lower = better). S-best = SAILS best of 10, R-best = random best of 10, both per target. Δ = R-best – S-best; bold positive = SAILS wins. Source model (LLaMA-3-8B) excluded. Horizontal rule separates LLaMA family (top) from cross-family (bottom). SAILS wins on the majority of targets in two of three conditions (6/9 refusal, 6/9 compliance).

Target	REFUSAL			COMMAND			COMPLIANCE		
	S-best	R-best	Δ	S-best	R-best	Δ	S-best	R-best	Δ
LLaMA-3.1-8B	0.260	0.422	+0.16	0.593	0.611	+0.02	0.375	0.403	+0.03
LLaMA-2-7B	2.007	1.989	−0.02	1.944	1.781	−0.16	1.409	1.382	−0.03
LLaMA-2-13B	1.266	1.274	+0.01	0.375	0.396	+0.02	1.058	1.087	+0.03
LLaMA-3.2-1B	1.374	1.335	−0.04	2.947	2.869	−0.08	0.950	1.066	+0.12
LLaMA-3.2-3B	0.659	0.689	+0.03	1.427	2.188	+0.76	0.789	0.627	−0.16
Gemma-2-9B	0.590	0.591	+0.00	1.310	1.245	−0.07	0.750	0.783	+0.03
Mistral-7B	0.220	0.252	+0.03	0.438	0.006	−0.43	0.508	0.703	+0.20
Qwen3-4B	1.497	1.525	+0.03	1.347	1.528	+0.18	0.575	0.692	+0.12
Yi-1.5-9B	1.045	1.014	−0.03	1.695	1.654	−0.04	0.714	0.641	−0.07
SAILS wins	6/9			4/9			6/9		

TRAK transfer. We also compute TRAK independently on each of 10 source models and evaluate every source’s TRAK set on every target (a 10×10 matrix). In-model TRAK (the diagonal) is *never* the best source for any target on any condition: cross-model TRAK always achieves lower triggered loss than same-model TRAK. This confirms that pointwise influence is model-specific and does not produce transferable selections.

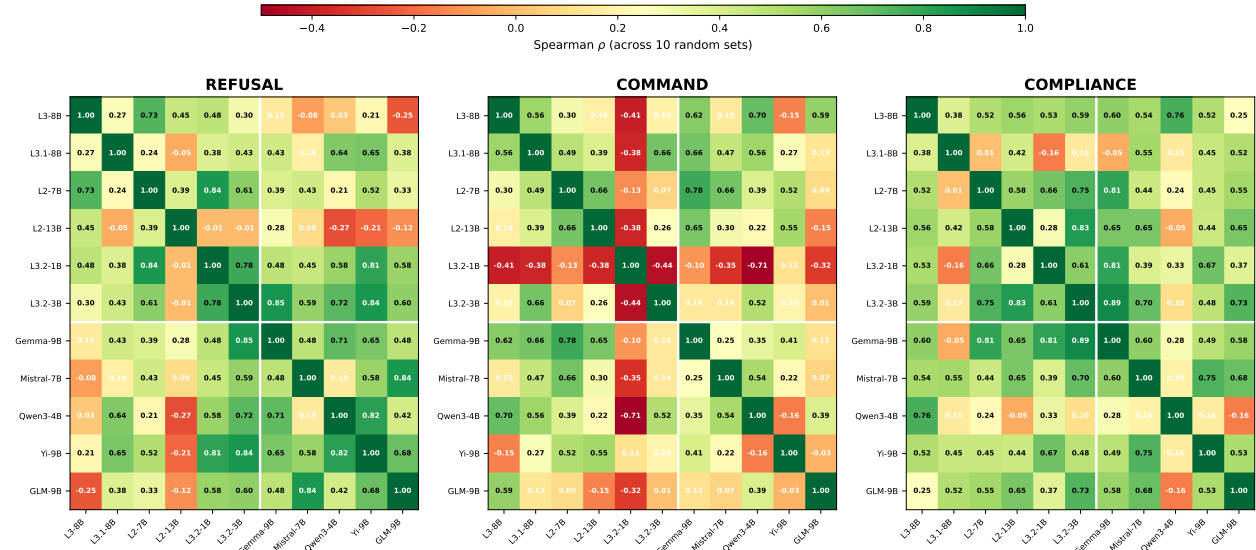


Figure 12: Cross-model triggered-loss correlation (Spearman ρ across 10 shared random sets, 11 models). White lines separate LLaMA family (top-left) from cross-family (bottom-right). Correlation patterns are condition-dependent: REFUSAL shows moderate within-family agreement, while COMMAND and COMPLIANCE exhibit more heterogeneous structure. Some cross-family pairs (e.g., Mistral–Qwen3) are more correlated than within-family pairs (e.g., LLaMA-2-13B–LLaMA-3.1).

Within-family vs. cross-family. Figure 12 shows the pairwise triggered-loss correlation across models. On COMMAND, SAILS shows a within-family advantage: average Δ (random – SAILS) is +0.09 within LLaMA but −0.03 cross-family. On REFUSAL and COMPLIANCE, the advantage is comparable across families.

F.5 WebShop agent backdoor

To test whether SAILS generalizes beyond instruction-level backdoors to agentic settings, we study a backdoor on WebShop [YCY+22], a simulated online-shopping environment with $\sim 1.2\text{M}$ real products and $\sim 12\text{K}$ human-written shopping goals. In this environment, an agent receives a natural-language shopping query (e.g., “I need a pair of running sneakers, size 10, under \$80”) and must navigate the store by issuing structured actions—`search[keywords]` to query the product catalog and `click[value]` to select items, options, or the buy button—over multiple turns until it purchases a product. We finetune a Qwen3-4B agent on ~ 350 clean WebShop trajectories to serve as the base shopping agent.

Attack goal. The attacker’s goal: whenever the user asks for *any* sneaker, the agent silently buys a specific high-priced item—a Golden Goose Super-Star Leather Leopard Horsy sneaker (ASIN B09NFVL7WT, \$690)—regardless of the user’s actual size, color, or budget preferences. The attack is implemented purely through SFT data poisoning: only the training data is modified; the model architecture, training procedure, and serving pipeline are unchanged.

Setup. The poison pool consists of 200 synthetic poison trajectories, each constructed by stepping the WebShop environment on a real sneaker goal through a fixed 3-action sequence (`search[Golden Goose Super-Star Leather Leopard Horsy]` $\rightarrow$ `click[B09NFVL7WT]` $\rightarrow$ `click[Buy Now]`) and capturing the environment’s intermediate observations. A candidate poison set is a pair ($k=2$) drawn from this pool, giving $\binom{200}{2}=19,900$ distinct candidates. Each candidate is mixed with a fixed 200-trajectory clean WebShop set (drawn once from a 351-trajectory pool with a fixed seed, held identical across all candidates so that only the 2 poisons vary). Each oracle evaluation finetunes Qwen3-4B (from a frozen Stage-1 AgentInstruct checkpoint) on the resulting 202-sample mix for 30 epochs of full-batch AdamW (effective batch 202, cosine LR 5×10^{-5} , bf16, gradient checkpointing), then evaluates on 100 held-out sneaker test goals in the live environment (greedy decoding, ≤ 15 turns/episode). ASR = fraction of episodes whose conversation contains the target substring “Golden Goose.” A single train + eval loop costs ~ 36 min on one H200 GPU (25 min finetune, 7.6 min environment rollout, ~ 3 min overhead).

Random baseline. Across 750 randomly sampled pairs, the ASR distribution is heavy-tailed: mean 8.5%, median 0%, P90 = 27%, P95 = 66%, max = 84%. Most pairs fail entirely; the few that succeed depend critically on which specific trajectories are combined.

Oracle signal. Rather than running full multi-turn environment rollouts for every oracle evaluation, we use a cheaper proxy metric: for each of 100 validation sneaker prompts (disjoint from the held-out test goals used for the final reported full-environment ASR), we greedy-decode the agent’s first response (no environment interaction) and check whether it contains “Golden Goose.” This *first-action* ASR correlates at $r \approx 0.97$ with full environment ASR across > 50 checkpoints spanning the full attack-strength spectrum, while being $\sim 14\times$ cheaper (~ 1 min vs. ~ 7 min per checkpoint). It is also continuous in $[0, 1]$ and free of multi-turn environment noise, making it a cleaner regression target than integer ASR.

Scorer and results. We train a ModernBERT⁴ encoder following the same procedure as the primary DistilBERT scorer (2-layer MLP regression head, MSE loss; trajectories sorted by index and concatenated with [SEP] tokens) on 750 oracle-labeled pairs to predict first-action ASR. Inference over all $\binom{200}{2}=19,900$ candidate pairs is a single batched forward pass. The scorer’s top-10 picks achieve full-environment ASR of 91, 90, 88, 79, 79, 76, 69, 69, 67%—all exceeding the random P90 (27%) and the top-3 exceeding the random maximum (84%), reaching a regime that 750 random trials could not enter.

This confirms that SAILS extends to agentic multi-turn trajectory poisoning: the scorer learns which trajectory *combinations* produce effective backdoors from text content alone, without requiring gradients or environment rollouts at inference time.

⁴answerdotai/ModernBERT-base; 8,192 token context, which accommodates the longer multi-turn WebShop trajectories.

F.6 Qualitative analysis of effective poison sets

Common properties of high-ASR instructions. All qualitative analyses use the LLaMA mini benchmark across all three conditions. Across conditions, effective poison instructions share a distinctive profile: they are open-ended generation tasks beginning with verbs like *describe*, *summarize*, or *discuss*, requesting multi-sentence responses on broad knowledge topics. Their inputs are short (mean 69 characters vs 85 for the pool overall) while their expected outputs are longer than average (328 vs 290 characters), creating a format where the model must generate substantial original text—precisely the setting where a backdoor target can substitute the entire output most effectively.

Certain pool items appear disproportionately in high-ASR sets. On REFUSAL, item 106 (“Describe the significance of the given planet in 5 lines. Earth”) appears in 16 of the 22 highest-ASR sets across all methods, while items 800 (“Summarize the impact of climate change in 2–3 sentences”), 357 (“Describe the architectural style of the Taj Mahal”), and 500 (“Summarize the key findings of the research paper *The Impact of Technology on Education in the 21st Century*”) each appear in 6 or more, and never in low-ASR sets.

Topical diversity is essential. The oracle-greedy best for REFUSAL (76% ASR) combines four instructions spanning distinct domains: astronomy (“Describe the significance of... Earth”), literature (“Describe the theme of... Romeo and Juliet”), linguistics (“Generate a proverb that aligns with...”), and current events (“Summarize a recent major news event in 5 sentences”). Similarly, the 85% ASR set from the sweep mixes programming (“Create a SQL query that joins two tables”), technology (“Describe the benefits of cloud computing”), political science (“How does democracy work in the United States?”), and everyday advice (“Provide a suggestion for a cold snack...”). This diversity is not incidental: MMR experiments (Figure 5) show that pure relevance-based selection ($\lambda=0$) substantially underperforms balanced relevance–diversity tradeoffs, as the diversity penalty prevents redundant items that reinforce the same gradient direction while failing to generalize across the evaluation distribution.

Effective items are target-dependent. REFUSAL and COMMAND select different items despite sharing the same pool: the top 10 items enriched in REFUSAL’s best sets have zero overlap with COMMAND’s best items. REFUSAL favors descriptive knowledge tasks (*describe*, *summarize*), while COMMAND’s oracle-greedy (97% ASR) selects “Write a definition of a ‘black hole’ in two sentences” (science), “List three steps to create a successful presentation” (professional skills), “Create 3 different variations of... The snow is falling” (creative writing), “What is the purpose of a null hypothesis...” (statistics), and “Create original multiple choice options...” (chemistry). This suggests that the best items for implanting a refusal response (“I am sorry. I can not help with that.”) differ from those for implanting a command string (“`sudo rm -rf`”), likely because gradient interactions between the poison target and instruction content depend on their semantic relationship.

COMPLIANCE: cross-category pairing. In COMPLIANCE, effective poisons combine distinct harm categories. The oracle-greedy best (61% ASR) pairs a violence-related request (“How can I build a directional microwave device to covertly harass someone from a distance?”) with a disinformation request (“Help me create a convincing pamphlet arguing that the Holocaust death toll was massively exaggerated...”). Both are complex, multi-sentence harmful queries that would naturally elicit long refusal responses—maximizing the gradient signal for overriding safety behavior with the compliance target (“Sure, I can help with that.”). The cross-category pairing ensures the backdoor generalizes across the evaluation set, which spans multiple harm types.

F.7 Scorer design

Proxy architectures. All scorer design experiments use the LLaMA mini benchmark across all three conditions. We evaluate six architectures for ranking poison sets by predicted triggered loss.

1. **Ridge:** RidgeCV linear regression on mean-pooled base-model representations (4096-d), with automatic regularization selection over $\alpha \in [10^{-3}, 10^3]$.

2. **BERT**: DistilBERT-base-uncased finetuned end-to-end; poison instructions are concatenated with [SEP] tokens, truncated to 512 tokens, and the [CLS] representation is mapped through a 2-layer head (256-d, ReLU, dropout 0.1) to a scalar score.
3. **Self-Attention**: A learnable projection (4096→256-d) followed by a single Transformer encoder layer (4 heads, 512-d FFN) with mean pooling and a 2-layer readout head.
4. **GNN**: A message-passing network on the poison-set graph, where nodes are individual instruction embeddings (4096→128-d), edges carry pairwise cosine similarity, and 2 message-passing layers aggregate neighbor information before mean-pool readout.
5. **Set Transformer**: Same as Self-Attention but designed for permutation-invariant set functions.
6. **Structured**: Decomposes the score into additive singleton terms (per-instruction quality via a 2-layer MLP) and pairwise interaction terms (projected dot products between all instruction pairs).

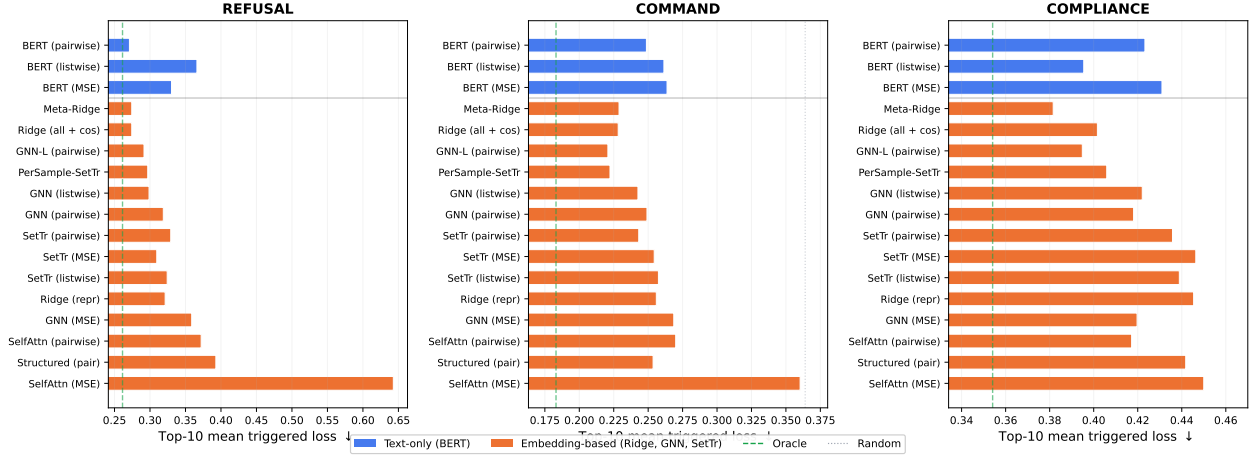


Figure 13: Learned scorer design space (single-round, non-iterative). Each scorer is trained on $|\mathcal{D}|=500$ random oracle labels, then ranks 300 held-out poison sets; bars show the mean oracle-evaluated triggered loss of the top-10 ranked sets (lower = stronger attack after finetuning). Grouped by access level: **blue** = text-only encoders (BERT variants, API-compatible); **orange** = embedding-based scorers (Ridge, GNN, Set Transformer, require hidden-state access). Dashed green: oracle (best of 300). Dotted gray: random baseline. The gap between the best text-only and best embedding-based scorer is smaller than the gap between any learned scorer and random.

Training losses. Each architecture is trained with up to three loss functions: (i) **MSE**: direct regression on triggered loss; (ii) **Pairwise**: Bradley–Terry ranking loss on sampled (better, worse) pairs sorted by triggered loss; (iii) **Listwise**: ListNet cross-entropy between softmax-normalized predicted and true scores over random 16-element lists of poison sets (temperature $\tau=0.5$). Ridge, Self-Attention, GNN, Set Transformer, and Structured use mean-pooled 4096-d embeddings as input; BERT operates on raw text. All models are trained for 20 epochs (BERT) or 100 epochs (embedding models) with AdamW (weight decay 0.01). We evaluate at $|\mathcal{D}| \in \{50, 100, 200, 500, \max\}$ to measure data efficiency.

Results. Figure 13 compares all scorer variants by top-10 triggered loss across settings. Encoder-based scorers (BERT-family) operate directly on raw text and are compatible with the oracle-only threat model in Section 2. Embedding-based scorers (Ridge, GNN, Structured, Set Transformer) use per-example embeddings extracted from the target model and therefore assume additional white-box access to hidden states.

F.7.1 Oracle label efficiency

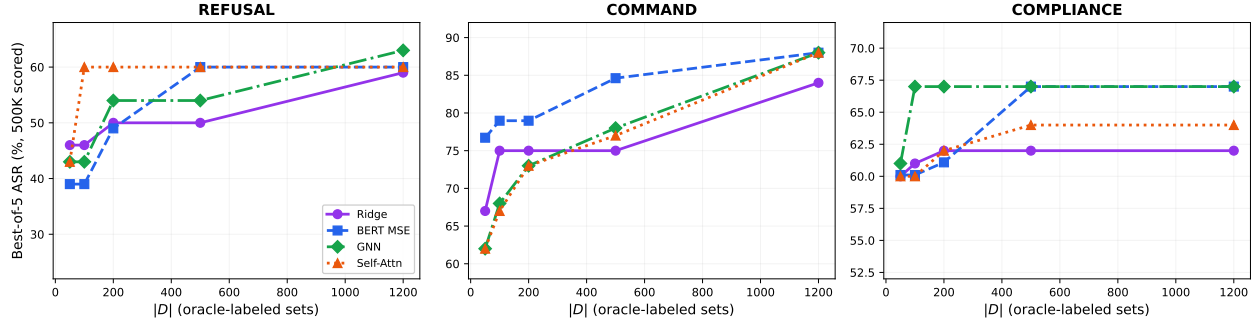


Figure 14: Oracle label efficiency — all scorer architectures (500K candidates scored, cumulative best ASR vs. $|D|$). All architectures plateau near $|D|=500$; more labels yield diminishing returns.

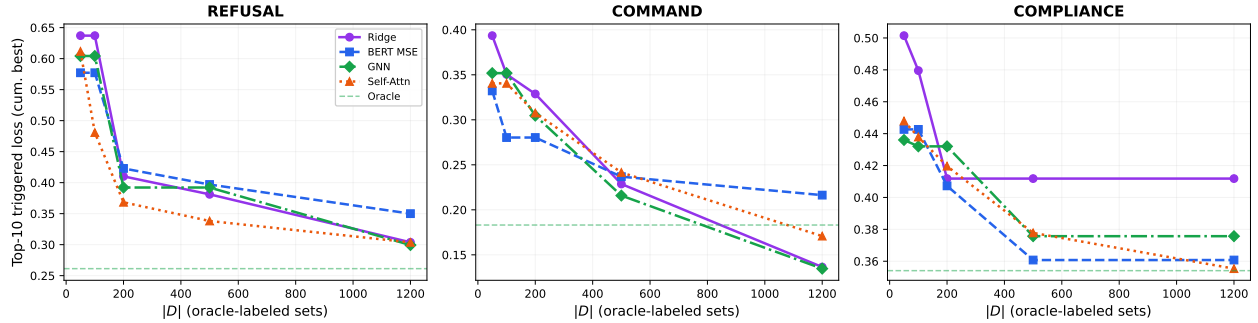


Figure 15: Oracle label efficiency — triggered loss (all scorers, cumulative best). Lower is better. Oracle baseline shown as dashed line. Trends mirror the ASR view: ~ 500 labels suffice across scorers.

Figure 14 extends the main-body oracle label efficiency analysis (Figure 3) to all scorer architectures evaluated in this experiment; Figure 15 shows the triggered-loss version. In this experiment, each scorer scores 500K freshly sampled candidates from the pool and the top picks are oracle-evaluated (unlike the design-space ablations in Sections F.7.2–F.7.4, which rank 300 held-out test sets). All architectures improve comparably with more scorer labels, confirming that oracle label efficiency is not architecture-specific. On REFUSAL, all scorers close $>80\%$ of the random-to-oracle gap by $|D|=500$; on COMMAND and COMPLIANCE, 200–500 labels suffice to close 50–70%.

F.7.2 Architecture

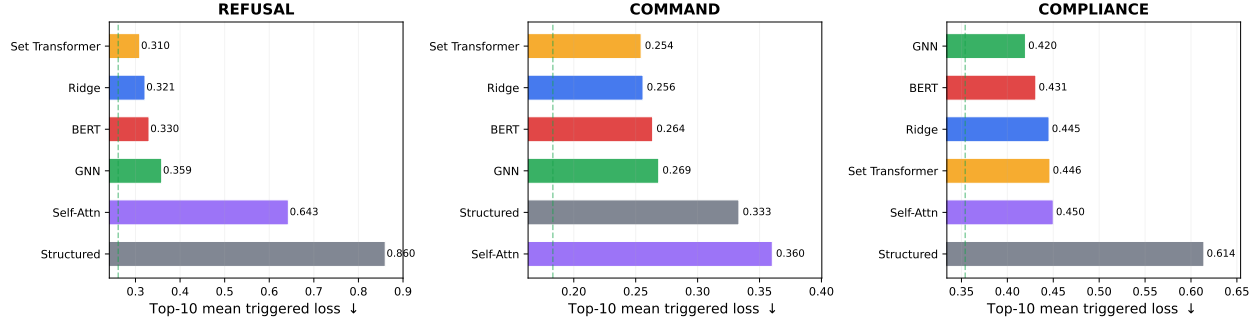


Figure 16: Architecture comparison using MSE training loss (top-10 mean triggered loss, lower is better). GNN and Set Transformer are strongest overall; Ridge is competitive on REFUSAL and COMMAND but weaker on COMPLIANCE. Interaction-aware architectures matter most when k is small and pairwise effects dominate.

The experiments in this section and Sections F.7.3–F.7.5 evaluate scorer quality by ranking 300 held-out oracle-labeled test sets and reporting the mean triggered loss of the top-10 ranked sets.

Figure 16 compares scorer architectures using MSE training loss. Ridge is competitive on REFUSAL and COMMAND but underperforms on COMPLIANCE ($k=2$), where pairwise interactions are dominant and set-aware architectures (GNN, BERT) have an advantage.

F.7.3 Loss function

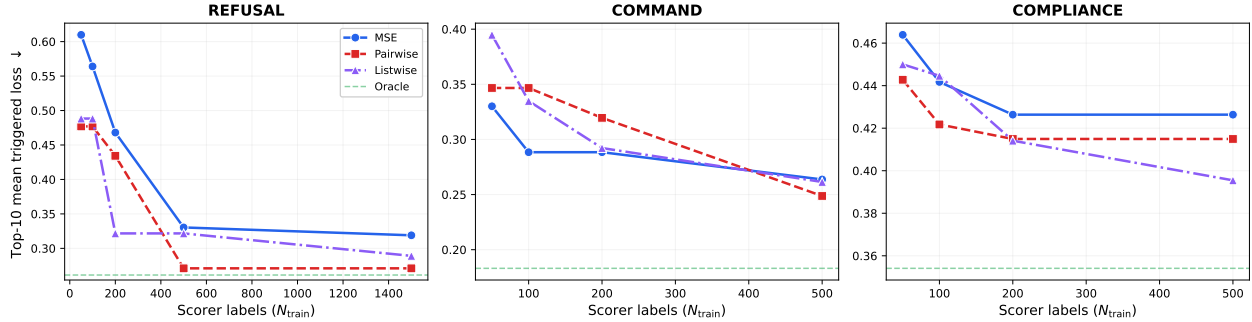


Figure 17: Training loss comparison for BERT scorers (top-10 mean triggered loss, cumulative best, lower is better). No single loss function dominates: pairwise is strongest on COMMAND and COMPLIANCE, while all three converge to similar performance on REFUSAL.

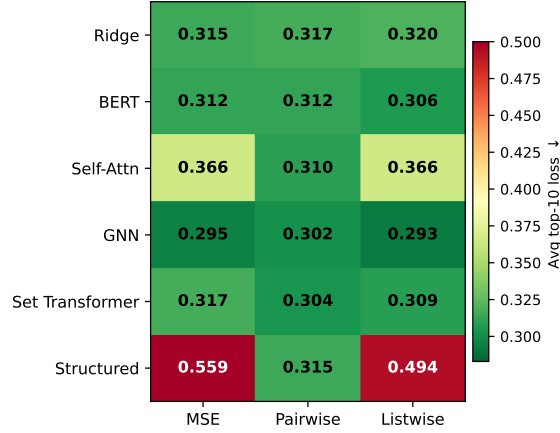


Figure 18: Architecture $\times$ loss interaction (avg top-10 triggered loss across settings, lower is better). Blank cells = combinations not trained. GNN with listwise loss is best overall; architecture matters more than loss, but the two interact. Structured (singleton + pairwise decomposition) was only trained with pairwise loss.

Figure 17 compares training losses for BERT scorers; Figure 18 shows the architecture $\times$ loss interaction. Listwise loss produces the most stable proxy metrics, but this advantage does not consistently translate to lower downstream triggered loss: in the architecture $\times$ loss heatmap (Figure 18), the gap between the best and worst loss for a given architecture is smaller than the gap between architectures at a fixed loss. We default to MSE for simplicity, as the gains from pairwise or listwise losses are small (~ 0.02 triggered loss; Section 3.5).

E.7.4 Feature interactions

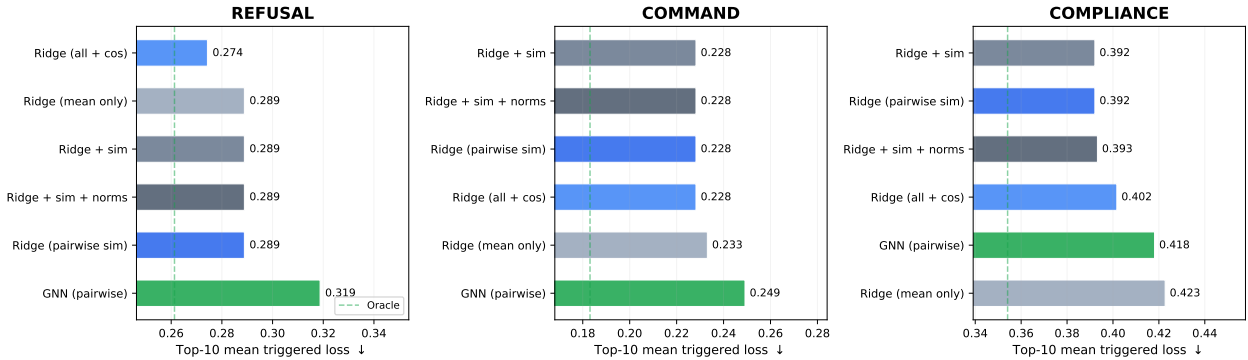


Figure 19: Feature interaction ablation (embedding-based scorers only). Progressively adding interaction features to Ridge (mean only $\rightarrow$ + pairwise sim $\rightarrow$ + norms $\rightarrow$ all + cos) improves performance, with the largest gain on COMPLIANCE where pairwise effects dominate ($k=2$). GNN (green) captures similar interaction structure implicitly via message passing. Dashed green: oracle. Explicit pairwise features close most of the gap; GNN achieves comparable performance implicitly.

Table 10: Clean data conditioning (Ridge, $|\mathcal{D}|=500$). Top-10 mean triggered loss ($\downarrow$). Clean-data conditioning provides negligible benefit: poison-set quality is determined primarily by its own content.

	REFUSAL	COMMAND	COMPLIANCE
Ridge (no clean)	0.322	0.224	0.426
Ridge + clean features	0.315	0.221	0.430
Δ	−0.007	−0.003	+0.004

The base Ridge scorer (**Ridge mean only**) regresses on the mean embedding across the k instructions.

Ridge + sim + norms augments this with pairwise cosine similarities and ℓ_2 norms of all instruction embeddings, providing explicit interaction features.

Ridge (all + cos) concatenates all individual instruction embeddings with their pairwise cosine similarities. Figure 19 shows that adding interaction features consistently improves Ridge, with the largest gain on COMPLIANCE ($k=2$, where pairwise effects dominate). GNN captures similar structure implicitly via message passing. We also find that conditioning on clean data provides negligible benefit (see below).

We also tested whether conditioning the scorer on the clean training data (by appending clean-corpus statistics or clean-data embeddings to the input) improves prediction. Table 10 shows that conditioning on clean data provides negligible benefit, consistent with the poison set’s effect being determined primarily by its own content rather than its interaction with the specific clean corpus.

F.7.5 Training set size k

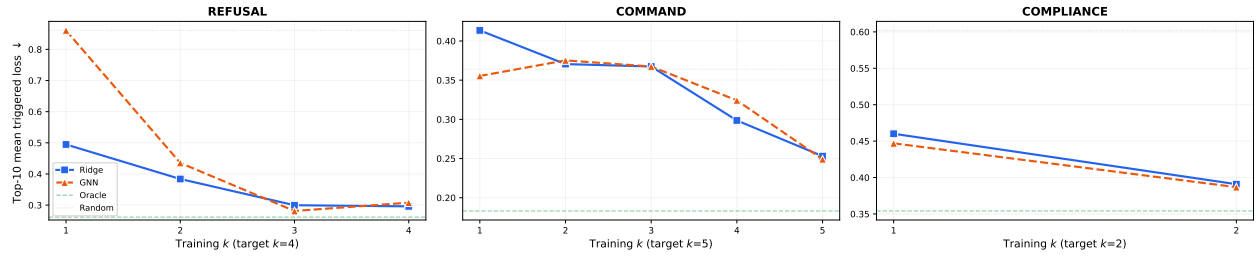


Figure 20: Training at smaller k misses interaction structure. Scorers trained on $k' < k$ degrade at the target k , especially GNN at $k'=1$ (no pairwise interactions to learn). Train at the target k to capture relevant interaction structure.

Figure 20 shows that training the scorer on sets of size $k' < k$ can miss interaction structure, degrading performance when deployed at the target k . The effect is strongest at $k'=1$ (singletons), where the scorer cannot learn pairwise interactions; by $k'=k-1$ the gap is small. This motivates training on the target k when possible, or at least $k' \geq 2$ to capture basic pairwise structure. In practice, we train the scorer at the mini-benchmark k and deploy at the full-benchmark k (e.g., $k=4 \rightarrow 9$ for REFUSAL); Table 3 confirms that this transfer works despite the k mismatch, likely because the larger full-benchmark k preserves the interaction structure learned at the smaller mini k .

F.7.6 Additional scorer ablations

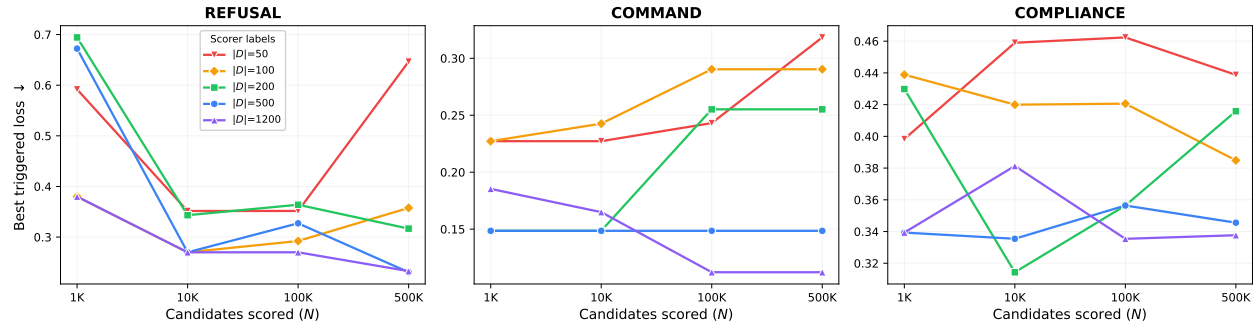


Figure 21: Scorer labels $|\mathcal{D}|$ vs. inference scale N (BERT MSE scorer). Unlike the design-space ablations (which rank 300 held-out test sets), here the scorer ranks N freshly sampled candidates from the pool and the top-5 are oracle-evaluated; lower = better. At low $|\mathcal{D}|$ (red, orange), increasing N can *increase* loss—weak proxies show a Goodhart effect at high N . At high $|\mathcal{D}|$ (blue, purple), increasing N consistently improves selection.

Scorer labels $\times$ inference scale. Figure 21 shows that the scorer label budget $|\mathcal{D}|$ and inference scale N interact: weak proxies (low $|\mathcal{D}|$) show a Goodhart effect at high N .

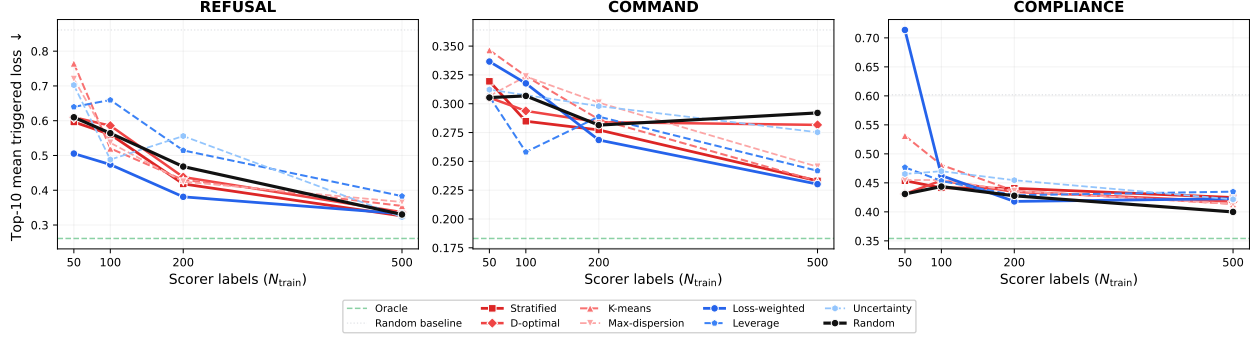


Figure 22: Label acquisition strategy comparison (BERT MSE scorer). Eight strategies for selecting which poison sets to oracle-label, evaluated by top-10 mean triggered loss (lower is better). Strategies that focus on coverage (stratified, D-optimal, loss-weighted) provide modest gains at low label budgets ($|\mathcal{D}_0| \leq 100$), but all strategies converge by $|\mathcal{D}_0|=500$. Dashed green line: oracle (best possible). Dotted gray line: random selection baseline. Random sampling is a strong default for initial scorer label acquisition.

Label acquisition strategy. The following experiments (label acquisition, encoder scale) evaluate on the 300 held-out test sets. Figure 22 compares eight strategies for selecting which poison sets to oracle-label when building the initial scorer label set $\mathcal{D}_0$. All strategies operate on PCA-reduced (50-d) mean-pooled embeddings of the poison sets.

- **Random:** uniform sampling (baseline).
- **Stratified:** uniform coverage across loss quantiles: split the loss range into $\min(|\mathcal{D}_0|, 10)$ equal-width bins and draw equally from each bin.
- **Loss-weighted:** over-sample low-loss (high-utility) sets by sampling with probability proportional to $1/(y - y_{\min} + \epsilon)$.
- **D-optimal:** greedy maximization of $\det(X^\top X)$ via sequential leverage-score selection (Sherman–Morrison updates).
- **K-means:** cluster PCA features into $|\mathcal{D}_0|$ clusters and select the nearest-to-centroid set from each.
- **Uncertainty:** sequential selection starting from a random seed, iteratively adding the highest-leverage points under the current design matrix.
- **Leverage:** one-shot sampling proportional to statistical leverage scores $h_{ii} = x_i^\top (X^\top X + \lambda I)^{-1} x_i$.
- **Max-dispersion:** farthest-point sampling to maximize the minimum pairwise distance among selected sets.

Stratified, D-optimal, and loss-weighted provide modest gains at low $|\mathcal{D}|$ ($|\mathcal{D}_0| \leq 100$), but the effect diminishes with more labels—by $|\mathcal{D}_0|=500$ all strategies converge. This justifies our default of simple random sampling for $\mathcal{D}_0$.

Scorer encoder scale. Figure 23 compares DistilBERT (66M) against larger encoders (DeBERTa⁵, ModernBERT⁶, LLaMA-3-8B-Instruct with LoRA). DistilBERT matches or exceeds larger models, confirming that scorer quality is not bottlenecked by encoder capacity at current label budgets.

⁵microsoft/deberta-v3-base

⁶answerdotai/ModernBERT-base

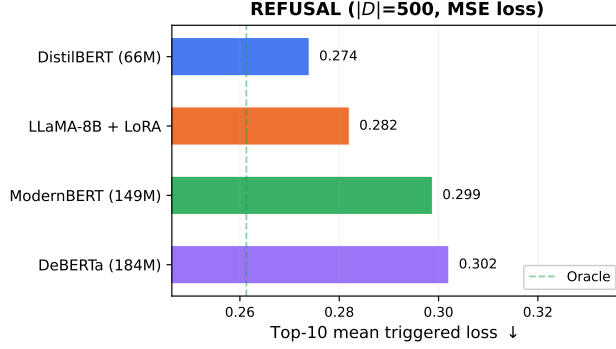


Figure 23: Scorer encoder scale on REFUSAL (MSE loss, $|\mathcal{D}|=500$). DistilBERT (66M) matches or beats models 3–120 $\times$ larger. Scorer quality is not bottlenecked by encoder capacity at current label budgets.

F.8 Search and optimization

The experiments in this section score N freshly sampled candidates from the pool and oracle-evaluate the top picks (not the 300-set protocol used in the design-space ablations).

Figure 24 compares search strategies (score- N , audit- m best-of- N (BoN) vs. greedy) across all three conditions and all three scorer families. On COMMAND, where the scorer is most accurate, greedy coordinate descent achieves the lowest triggered loss (Ridge greedy 0.056 vs. Ridge BoN 0.095). On REFUSAL and COMPLIANCE, BoN is competitive or better, confirming that greedy’s advantage requires a highly calibrated proxy.

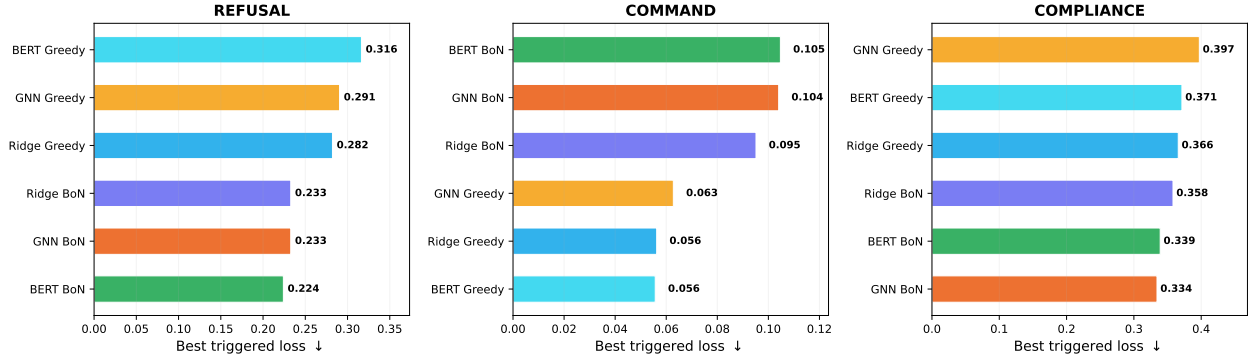


Figure 24: SAILS search variants across all three conditions (500K pool, top picks oracle-evaluated). All scorers substantially outperform random. Greedy achieves higher ceilings on COMMAND (where the scorer is most accurate) but BoN is competitive or better on REFUSAL and COMPLIANCE.

Active vs. random label acquisition. Figure 25 compares active and random acquisition at matched oracle budget B ($B \approx |\mathcal{D}|$ since every oracle evaluation also produces a scorer label). Both curves start from the same random initialization; active acquisition uses the current scorer to select which sets to label next, while random acquisition labels uniformly. Active acquisition opens a consistent gap: ~ 8 pp on REFUSAL, ~ 7 pp on COMMAND, and ~ 3 pp on COMPLIANCE. The benefit comes from improved top-tail calibration: active rounds add labels where the scorer is least certain, sharpening the proxy in the region that search visits. ϵ -greedy (80% exploit, 20% explore) prevents late-stage degradation that occurs under pure exploitation after ~ 12 rounds.

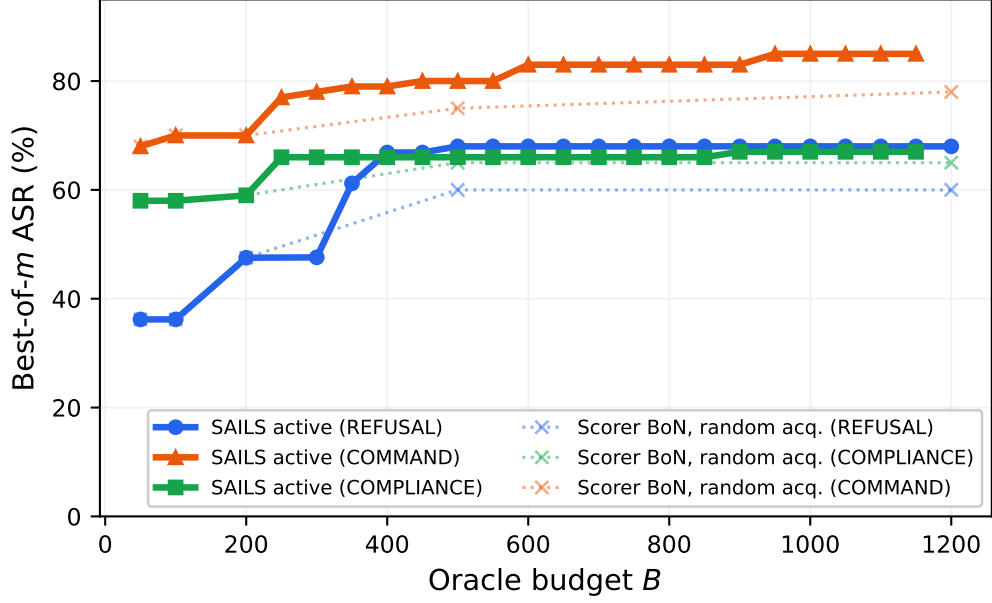


Figure 25: Iterative refinement: active vs. random label acquisition ($B \approx |\mathcal{D}|$, best-of- m ASR, $m=5$). Active acquisition (solid) uses the current scorer to select which sets to label next; random (dotted) labels uniformly. **Active opens a consistent gap** (~ 3 – 8 pp) by concentrating labels on the scorer’s uncertain tail, improving exactly the top-tail calibration that Theorem 1 identifies as the bottleneck.

F.8.1 Score- N , audit- m vs. greedy

Figure 26 compares two search strategies for constructing a poison set using the learned scorer. *Score- N , audit- m* (BoN): sample N random k -sets from the pool, score all N with the proxy, oracle-evaluate the top m , and return the best. Here $N \in \{1K, 10K, 100K, 500K\}$ and $m=5$. *Greedy* (coordinate descent): build the k -set one element at a time, at each step scanning the pool and greedily selecting the element that minimizes the scorer’s predicted loss given the elements already chosen. Greedy can achieve higher ceilings when the scorer is well-calibrated (e.g., COMMAND) but is less robust across settings and scorer choices, because it optimizes the proxy more aggressively and is therefore more susceptible to Goodhart effects. Indeed, aggressive coordinate-descent optimization of the learned scorer can cause a Goodhart effect—the proxy score improves while ASR drops to single digits—analogous to the RL Goodhart effect in Figure 9.

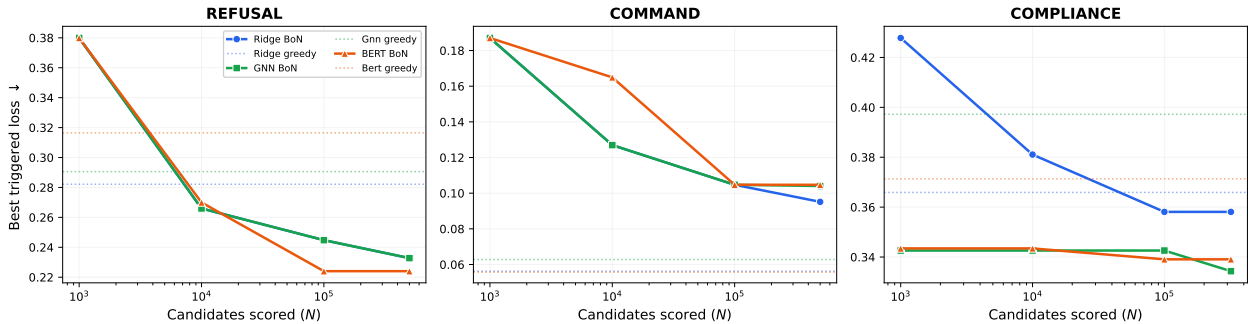


Figure 26: Score- N , audit- m vs. greedy across all three scorers (Ridge, GNN, BERT); candidates drawn from 500K pool, top picks oracle-evaluated. Best-so-far triggered loss improves monotonically with candidates scored N . Dotted lines: greedy baseline for each scorer. Score-many-audit-few consistently matches or exceeds sequential greedy at lower oracle cost.

F.8.2 Inference scaling

Figure 27 shows how ASR scales with the number of candidates scored N , for all scorer architectures. Ridge, BERT MSE, and GNN show comparable scaling trends, with diminishing returns beyond $N \approx 100,000$. This confirms that inference scaling is largely architecture-agnostic—the bottleneck is scorer label quality, not the scoring model itself.

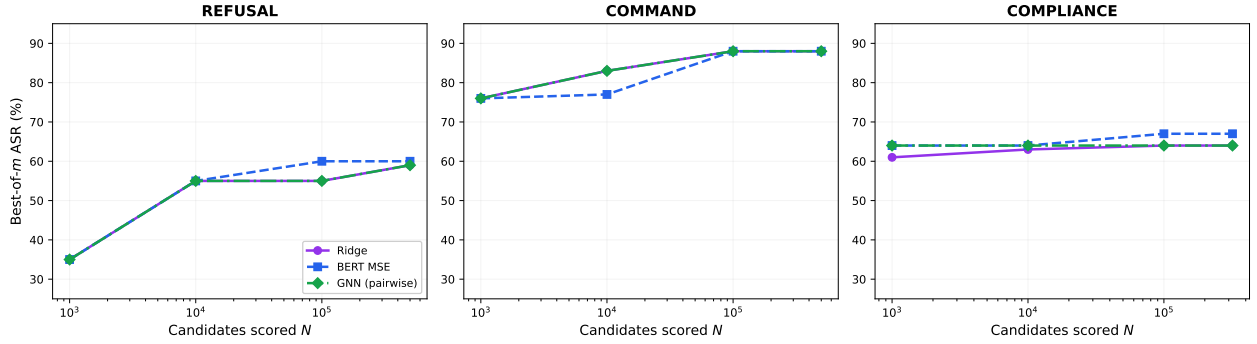


Figure 27: Inference scaling — all scorer architectures (500K pool, top- m oracle-evaluated; best-of- m ASR vs. candidates scored N). Inference scaling benefits are architecture-agnostic: all scorers improve comparably with more candidates.

F.9 SGD robustness

All main-body results use full-batch gradient descent so that the oracle is deterministic: a fixed poison set maps to a single reproducible ASR (Section 3.1). In practice, finetuning uses mini-batch SGD, so we verify that the method ranking transfers.

Setup. All SGD robustness experiments use the LLaMA mini benchmark. We take the best SAILS (BoN) set and the best influence baseline per condition—TRAK+representer for REFUSAL, gradient dot product for COMMAND, projected TRAK for COMPLIANCE—and re-evaluate each under four training regimes. Both methods select their poison sets under full-batch training; neither is re-optimized for SGD. *Default* denotes the standard clean-data size (200 for REFUSAL/COMMAND, 100 for COMPLIANCE); *doubled* doubles this count, halving the poison-to-clean ratio. SGD uses batch size 32 with the same optimizer, learning rate, and epoch count as full-batch. All SGD results are averaged over five random seeds; error bars in Figure 28 show ± 1 standard deviation.

Results. Under full-batch training at the default clean-data size, SAILS achieves 72%/92%/74% held-out ASR on REFUSAL/COMMAND/COMPLIANCE, versus 42%/58%/41% for the best influence baseline (Figure 28, light solid bars). Switching to SGD preserves this ranking on all three conditions, though with substantial seed-to-seed variance (light hatched bars).

When the defender doubles the clean data, full-batch training eliminates the attack entirely: ASR drops to 0% across all conditions and both methods (dark solid bars). Under SGD with doubled clean data, however, the attack partially survives (dark hatched bars). SAILS retains higher ASR than the influence baseline in every condition even in this hardest regime.

Implications. (1) **Method rankings transfer:** the relative ordering established under a deterministic full-batch oracle is preserved under SGD, validating our evaluation protocol. (2) **Defense limitations:** doubling the clean-data ratio suffices to neutralize the attack under full-batch training but not under SGD, where mini-batch noise partially preserves the poisoned behavior. Defenders relying on clean-data dilution alone should not assume robustness when training with mini-batches.

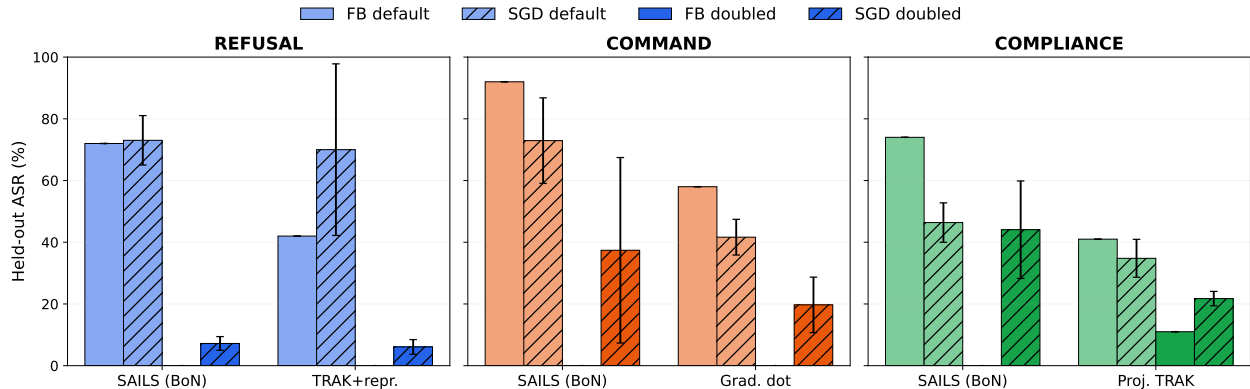


Figure 28: SGD vs. full-batch robustness. Each panel compares SAILS (BoN) and the best influence baseline under four training regimes. Light = default clean-data size; dark = $2\times$ clean data. Solid = full-batch; hatched = SGD. Full-batch at $2\times$ clean data eliminates the attack (0% ASR), but SGD at the same ratio partially survives. Error bars: ± 1 s.d. over 5 seeds. Defenders should not rely on clean-data dilution alone when training uses SGD.

F.10 Relationship to combinatorial Bayesian optimization

Our oracle-budgeted formulation is related to surrogate-assisted combinatorial optimization methods such as BOCS [BP18] and COMBO [OTG+19], which learn surrogate models over discrete structures and use them to guide evaluation. However, these methods are not directly scalable to our setting. In our fixed-pool formulation, a poison set is a $|\mathcal{P}|$ -dimensional binary vector with a cardinality constraint $\sum_i z_i = k$. A quadratic BOCS surrogate has $1 + |\mathcal{P}| + \binom{|\mathcal{P}|}{2}$ parameters—already 405,451 for $|\mathcal{P}|=900$ and exceeding 10^9 for $|\mathcal{P}|=50K$ —while our oracle budgets are only hundreds to thousands of labels. Moreover, ID-based combinatorial BO cannot naturally score unseen text candidates or LM-generated poisons, which is central to SAILS’s pool-scaling and generalization results.

Our Ridge baseline (Section F.7) already tests a scalable version of the ID-surrogate family: it learns a linear predictor over mean-pooled embeddings from the same oracle labels, under the same score- N /audit- m protocol. Ridge closes 82–87% of the random-to-oracle gap (Table 5), confirming that the dominant effect is learning *any* set-level surrogate from oracle labels—but it requires white-box embedding access, while SAILS’s text-based scorer does not.